itenas
Institut Teknologi Nasional
Bandung

# IFB-451
# BIG DATA

## TEACHING MODULES

**HEALTH INFORMATICS FOCUS**

FOR THE BACHELOR OF INFORMATICS PROGRAMME


**DEVELOPED BY**

**Dr Edi Nuryatno**

*PhD, MSc, MACS CT, BSc*

IP3P (Int.) | ICH GCP (Aus.) | SKT (Aus.) | CCI (Aus.) | GCP (Aus.) | AI Teach. (Int.)




# TABLE OF CONTENTS


Table of Contents....2
Syllabus Overview....7
Course Orientation....7
Learning Themes Across the Modules....7
Pedagogical Progression (Bloom's Taxonomy)....9
Healthcare-Oriented Applications....9
Expected Graduate Competence....10
Learning Outcomes Overview....11
Bloom's Taxonomy Perspective....11
Teaching Modules Overview....13
The Correlation between Teaching Modules based on SCs....13
Module 1 – Introduction to Big Data....15
Learning Outcomes....15
Theory Overview....15
Big Data Ecosystem....16
Lab Exercise — Exploratory Analysis with pandas....19
Sample Code — pandas (Exploratory - PySpark Word Count)....20
Case Study....21
Suggested Readings....21
Module 2 – Big Data Ecosystem and Architectures....22
Learning Outcomes....22
Theory Overview....22
Big Data Reference Architectures....23
Lab Exercise — Setting up Hadoop and Spark....24
Sample Code — PySpark Word Count (phyton)....25
Case Study....26
Suggested Readings....26
Module 3 – Data Storage and Management....27
Learning Outcomes....27
Theory Overview....27

Big Data Storage Architecture ..... 28
Lab Exercise — Working with HDFS ..... 29
Sample Code — PySpark with Parquet (phyton): ..... 29
Case Study ..... 30
Suggested Readings ..... 30
Module 4 – Hadoop Ecosystem ..... 31
Learning Outcomes ..... 31
Theory Overview ..... 31
Hadoop Ecosystem Components ..... 32
Lab Exercise — Running Hadoop and Hive ..... 32
Sample Code — MapReduce in Python (Hadoop Streaming) ..... 34
Case Study: ..... 35
Suggested Readings ..... 35
Module 5 – MapReduce Programming ..... 36
Learning Outcomes ..... 36
Theory Overview ..... 36
MapReduce Data Flow ..... 37
Lab Exercise — Word Count in Java ..... 38
Sample Code — Hadoop Streaming with Python ..... 39
Case Study ..... 39
Suggested Readings ..... 40
Module 6 – Apache Spark Basics ..... 41
Learning Outcomes ..... 41
Theory Overview ..... 41
Spark Architecture ..... 42
Lab Exercise — Getting Started with PySpark ..... 43
Sample Code — Word Count with RDD ..... 44
Case Study ..... 44
Suggested Readings ..... 44
Module 7 – Spark SQL and DataFrames ..... 45
Learning Outcomes ..... 45
Theory Overview ..... 45

Spark SQL Workflow ..... 46
Lab Exercise — Spark SQL with DataFrames ..... 47
Sample Code — JSON + SQL Integration ..... 48
Case Study ..... 48
Suggested Readings ..... 49
Module 8 – Spark Streaming ..... 50
Learning Outcomes ..... 50
Theory Overview ..... 50
Spark Structured Streaming Workflow ..... 50
Lab Exercise — Streaming Word Count ..... 51
Sample Code — Kafka Integration ..... 52
Case Study ..... 53
Suggested Readings ..... 53
Module 9 – NoSQL Databases for Big Data ..... 54
Learning Outcomes ..... 54
Theory Overview ..... 54
Types of NoSQL Databases ..... 55
Lab Exercise — MongoDB Basics ..... 56
Sample Code — Spark + MongoDB Connector ..... 57
Case Study ..... 58
Suggested Readings ..... 58
Module 10 – Cloud Platforms for Big Data (AWS, GCP, Azure) ..... 59
Learning Outcomes ..... 59
Theory Overview ..... 59
Cloud Big Data Architecture ..... 60
Lab Exercise — Querying Data on BigQuery (GCP) ..... 60
Sample Code — AWS S3 with PySpark ..... 61
Case Study ..... 62
Suggested Readings ..... 62
Module 11 – Machine Learning with Big Data ..... 63
Learning Outcomes ..... 63
Theory Overview ..... 63

Big Data ML Workflow ..... 64
Lab Exercise — Logistic Regression with Spark MLlib ..... 64
Sample Code — K-Means Clustering ..... 65
Case Study ..... 66
Suggested Readings ..... 66
Module 12 – Deep Learning and Big Data (TensorFlow, PyTorch, Spark Integration) ..... 67
Learning Outcomes ..... 67
Theory Overview ..... 67
Deep Learning with Big Data Workflow ..... 68
Lab Exercise — Image Classification with TensorFlow ..... 68
Sample Code — PyTorch + Spark Integration ..... 69
Case Study ..... 70
Suggested Readings ..... 70
Module 13 – Data Visualization and Big Data Dashboards ..... 71
Learning Outcomes ..... 71
Theory Overview ..... 71
Big Data Dashboard Example ..... 72
Lab Exercise — Real-Time Dashboard with Grafana ..... 72
Sample Code — Python + Plotly Visualization ..... 73
Case Study ..... 73
Suggested Readings ..... 73
Module 14 – Ethics, Privacy, and Future Trends in Big Data ..... 74
Learning Outcomes ..... 74
Theory Overview ..... 74
Responsible Big Data Lifecycle ..... 75
Lab Exercise — Data Anonymization ..... 75
Case Study ..... 76
Suggested Readings ..... 76
Grading Systems ..... 77
Weighting Mark ..... 77
SCs and Assessment Mechanism ..... 77
Bibliography ..... 78

Authoritative Texts on Big Data & Ecosystems....78
NoSQL, Storage & Databases....78
Spark & Advanced Data Frameworks....78
Machine Learning, Deep Learning & AI in Healthcare....79
Visualization & Dashboards....79
Ethics, Governance & Policy....79
Healthcare Informatics & Big Data Applications....79
Cloud & Emerging Tech....80
Dr. Edi Nuryatno Contributions....80

# SYLLABUS OVERVIEW

## Course Orientation

This course equips students with the knowledge and practical skills required to manage and analyse large-scale, complex health datasets and translate them into meaningful information for clinical, public health, biomedical and health-system decision-making. Students' progress from the foundations of Big Data (SC1) through distributed storage and processing frameworks (SC2–SC4), cloud platforms and machine learning (SC5), and advanced analytics, visualisation and responsible data governance (SC6).

The Health Informatics focus emphasises that technical capability alone does not constitute clinical value. Students must also consider data quality, interoperability, workflow integration, explainability, privacy, security, human oversight and appropriate validation. The course prepares students for work in Health Informatics, MedTech, healthcare analytics, public health informatics and responsible AI.

## Learning Themes Across the Modules

**1. Foundational Concepts of Big Data in Healthcare (SC1)**

- Students examine the 5Vs of Big Data—Volume, Velocity, Variety, Veracity and Value—and their relevance to clinical and public health data.
- Examples include electronic health records (EHRs), medical imaging archives, laboratory results, wearable-device data, insurance claims and genomic repositories.
- Students explore batch, streaming, Lambda and Kappa architectural patterns and assess their suitability for different health-data scenarios.
- Emphasis is placed on distinguishing the presence of large or complex data from the ability to generate clinically meaningful and actionable information.

**2. Data Storage, Management & Processing Frameworks (SC2–SC3)**

- Students examine distributed storage technologies, including HDFS, cloud object storage, NoSQL databases and analytical file formats such as Parquet.
- They progress to Hadoop and Spark for distributed data processing, including applications such as:
    - Conducting population-level epidemiological analyses.
    - Processing clinical notes, laboratory data and pathology reports at scale.
    - Supporting disease-surveillance and health-service monitoring systems.
- Students examine schema design, data provenance, interoperability, fault tolerance and the appropriate selection of storage technologies.

- The CAP theorem is introduced as a trade-off between consistency and availability when network partitions occur, rather than as a simple choice of any two guarantees.
- Students also learn to distinguish wide-column databases such as Cassandra and HBase from analytical columnar formats such as Parquet and ORC.

**3. Advanced Big Data Engines & Real-Time Analytics (SC3–SC4)**

- Building on the MapReduce programming model, students transition to Apache Spark for distributed batch, iterative and interactive data processing.
- They develop practical skills in Spark SQL, DataFrames and Structured Streaming.
- Case studies include ICU monitoring pipelines, health-insurance claims analysis, hospital activity monitoring and outbreak surveillance.
- Students examine event-time processing, latency, data quality, checkpointing, fault tolerance and the management of incomplete or delayed data.
- Streaming analytics is presented as a mechanism that may reduce the time required to identify clinically significant events. Any claim of improved patient outcomes must be supported by appropriate clinical evaluation.

**4. Integrating Platforms, ML, and AI (SC5)**

- Students examine cloud-based services provided by AWS, Microsoft Azure and Google Cloud for scalable storage, distributed processing and analytics.
- They evaluate cloud and on-premises solutions in relation to scalability, cost, security, governance, data residency and vendor dependency.
- Students use Spark ML to develop baseline predictive and analytical models, including:
    - Risk-prediction models for conditions such as stroke or cardiovascular disease.
    - Hospital readmission and health-service utilisation models.
    - Clustering and anomaly-detection models for claims and operational data.
- Students develop reproducible machine-learning pipelines covering data preparation, feature engineering, model training, validation and evaluation.
- Explainability, calibration, class imbalance, bias, external validation and human oversight are treated as essential considerations for healthcare applications.
- Model development within the course does not, by itself, constitute approval for deployment in a clinical environment.

**5. Deep Learning, Visualization & Governance (SC6)**

- Students examine deep-learning applications in medical imaging, genomics and natural language processing of clinical text.
- TensorFlow and PyTorch are introduced for model development, while Spark may be used for distributed data preparation and appropriately supported scalable workflows.

- Students design dashboards and visualisations for clinicians, health-service managers, researchers and policymakers. Examples include hospital-capacity monitoring, regional epidemiological mapping and disease-surveillance dashboards.
- Visualisation activities emphasise accuracy, clarity, appropriate aggregation, uncertainty and fitness for the intended decision-making context.
- Students critically evaluate algorithmic bias, consent, privacy, security, data ownership, accountability and the broader social consequences of health-data use.
- Relevant legal frameworks include the EU General Data Protection Regulation, Indonesia's Personal Data Protection Law and, where jurisdictionally applicable, the United States Health Insurance Portability and Accountability Act.

## Pedagogical Progression (Bloom's Taxonomy)

- **Remembering and Understanding:** Students define and explain Big Data concepts, technologies and architectural patterns in health-related contexts.
- **Applying and Analysing:** Students use distributed storage, processing and analytical technologies to investigate health datasets and identify technical, data-quality and workflow issues.
- **Evaluating and Creating:** Students evaluate alternative technologies and architectures, critique existing Health Informatics solutions, and design end-to-end Big Data solutions incorporating analytics, visualisation, governance and ethical safeguards.

## Healthcare-Oriented Applications

Healthcare case studies are used throughout the course to connect technical learning with real decision-making requirements:

- **Public Health Informatics:** outbreak surveillance, immunisation monitoring and population health analysis.
- **Clinical Informatics:** clinical-data integration, streaming physiological data and predictive clinical decision support.
- **Biomedical Informatics and Research:** genomics, radiomics, medical imaging and clinical research data pipelines.
- **Health Systems Management:** service-demand analysis, claims analysis, resource allocation and operational dashboards.
- **Data Governance and Ethics:** privacy, consent, security, accountability, data residency and responsible AI governance.
- **Health Data Communication:** dashboards and visualisations designed for clinicians, managers, researchers and policymakers.

## Expected Graduate Competence

By course completion, students will be able to:

1. Design scalable health-data architectures for EHR, imaging, wearable, genomic and public health data.
2. Implement batch and streaming data-processing pipelines for appropriate clinical, operational and public health scenarios.
3. Select and apply suitable distributed storage, processing and analytical technologies.
4. Develop and evaluate machine-learning and deep-learning prototypes for predictive, analytical or decision-support purposes.
5. Assess data quality, explainability, bias, privacy, security, interoperability and governance requirements.
6. Distinguish a technical model or prototype from a clinically validated and deployable Health Informatics system.
7. Communicate analytical findings responsibly through clear dashboards, visualisations and decision-support outputs.

# LEARNING OUTCOMES OVERVIEW

## Bloom's Taxonomy Perspective

1. Remembering (Knowledge Recall)
   - Define key terms: Big Data, Data Lake, Hadoop, Spark, NoSQL.
   - List major characteristics of Big Data (Volume, Velocity, Variety, Veracity, Value).
   - Recall types of data analytics (descriptive, predictive, prescriptive).
2. Understanding (Comprehension)
   - Explain how Big Data differs from traditional data processing.
   - Summarize the Big Data pipeline from data ingestion to visualization.
   - Interpret Big Data use cases in health, finance, urban planning, etc.
   - Describe ethical and privacy issues in Big Data management.
3. Applying (Practical Implementation)
   - Use Hadoop or Spark to process large datasets.
   - Apply SQL/NoSQL queries to extract insights from structured/unstructured data.
   - Implement simple machine learning models on Big Data frameworks.
   - Utilize visualization tools (e.g., Tableau, Power BI, Python libraries) for Big Data storytelling.
4. Analysing (Critical Breakdown)
   - Differentiate between batch vs. real-time Big Data processing.
   - Compare performance of distributed computing frameworks (e.g., Hadoop vs. Spark).
   - Identify patterns, anomalies, and correlations in large datasets.
   - Break down challenges in Big Data security and governance.
5. Evaluating (Judgment & Decision-Making)
   - Critique the effectiveness of Big Data solutions in different industries.
   - Assess trade-offs between scalability, accuracy, and cost in Big Data systems.
   - Evaluate ethical implications of Big Data use (bias, fairness, privacy).
   - Recommend best-fit data architectures for specific real-world scenarios.
6. Creating (Innovation & Integration)
   - Design and prototype an end-to-end Big Data solution.
   - Develop algorithms for real-time analytics on streaming data.
   - Integrate Big Data with AI/IoT for smart city or healthcare applications.
   - Propose innovative strategies for sustainable and ethical Big Data governance.


BLOOM'S TAXONOMY FOR
BIG DATA LEARNING OUTCOMES
CREATING
Design and prototype an end-to and Big Data solution
• Develop algorithms for real-time analytics on streaming dat
• Integrate Big Data with oʃ/igƬ for smart city or healificare applications
• Propose innevative strategies for sustainable and ethical Big Data governance
EVALUATING
• Critique effectiveness of Big Data solutionsn different industries
• Assess trade-offs between scalability, accuracy, and cost in Big Data systems
• Recommen best fit data architectures for specific real-world scenarios
ANALYZING
• Differentiate between batch vs, real-time Big Data processing,
• Compare performance of distributed computing frameworks (e.g, Hadoop vs, Spark)
• Identify patterns, anomalies, and correlations in large datasets
• Break down challenges in Big Data security and governance
UNDERSTANDING
• Explain how Big.Data differs from traditional data, processing
• Summarize the Big Data pipeline from data ingestion to visuuilalization
• Interpret Big Data use cases in health, finance, and urban planning
REMEMBERING
• Define key terms: Big Data, Data Lake, Hadoop, .Spark, NoSQL
• List major characteristies of Big Data (e.g, volume, velocity-variety, veracicity,
• Recall types of data analytics (e.g, descriptive, predictive, prescriptive)

# TEACHING MODULES OVERVIEW

- Module 1: Introduction to Big Data: **SC1**
- Module 2: Big Data Ecosystem and Architectures: **SC1**
- Module 3: Data Storage and Management: **SC2**
- Module 4: Hadoop Ecosystem: **SC2**
- Module 5: MapReduce Programming: **SC3**
- Module 6: Apache Spark Basics: **SC3**
- Module 7: Spark SQL and DataFrames: **SC3**
- Module 8: Spark Streaming: **SC4**
- Module 9: NoSQL Databases for Big Data: **SC4**
- Module 10: Cloud Platforms for Big Data: **SC5**
- Module 11: Machine Learning with Big Data: **SC5**
- Module 12: Deep Learning and Big Data: **SC6**
- Module 13: Data Visualization and Big Data Dashboards: **SC6**
- Module 14: Ethics, Privacy, and Future Trends in Big Data: **SC6**

## The Correlation between Teaching Modules based on SCs

**SC1:** **Foundations:** This stage provides the conceptual base for all subsequent SCs.

- **Module 1**. Introduction to Big Data: establishes the motivation, definitions, and challenges.
- **Module 2**. Big Data Ecosystem and Architectures: show how the overall system is structured (data sources, ingestion, storage, processing, analytics).

**SC2:** **Storage & Processing Frameworks:** Storage (data persistence) directly supports processing frameworks like Hadoop.

- **Module 3**. Data Storage and Management: explores relational, distributed, and modern storage strategies.
- **Module 4**. Hadoop Ecosystem: introduces the batch-oriented processing model (HDFS + MapReduce + YARN).

**SC3:** **Core Processing Engines:** Spark builds on MapReduce concepts but improves performance; SQL/DataFrames bridge structured data with big data frameworks.

- **Module 5**. MapReduce Programming: teaches the fundamental batch computation model.
- **Module 6**. Apache Spark Basics: extends beyond MapReduce with faster in-memory computation.

- **Module 7**. Spark SQL and DataFrames: integrates structured queries for data manipulation.

**SC4:** **Advanced Data Handling:** Spark Streaming often integrates with NoSQL systems for ingesting and persisting streaming data.

- **Module 8**. Spark Streaming: adds real-time streaming capability.
- **Module 9**. NoSQL Databases for Big Data: supports flexible schema, scalable storage for unstructured/semi-structured data.

**SC5:** **Analytics & Platforms:** Cloud provides infrastructure; ML adds intelligence on top of processed data.

- **Module 10**. Cloud Platforms for Big Data: scalable infrastructure (AWS, GCP, Azure) for storage, compute, and managed services.
- **Module 11**. Machine Learning with Big Data: applied analytics using ML libraries within Spark/Cloud.

**SC6:** **Advanced Intelligence & Responsible Use:** Deep Learning and ML produce insights → Visualization translates them → Ethics ensures responsible usage.

- **Module 12**. Deep Learning and Big Data: applies neural networks for complex patterns (images, NLP, IoT data).
- **Module 13**. Data Visualization and Big Data Dashboards: communicates insights effectively to decision-makers.
- **Module 14**. Ethics, Privacy, and Future Trends in Big Data: integrates governance, ethical considerations, and emerging directions.

# MODULE 1 – INTRODUCTION TO BIG DATA

## Learning Outcomes

- Define Big Data and explain the 5V characteristics (Volume, Velocity, Variety, Veracity, Value).
- Differentiate Big Data systems from traditional database systems and identify common use cases.
- Describe the Big Data lifecycle and core components of a Big Data ecosystem.
- Perform basic exploratory analysis on a large dataset using Python/pandas and PySpark (overview).
- Identify major challenges (scalability, privacy, governance) when working with Big Data.

## Theory Overview

What is Big Data?

Big Data refers to datasets whose size or complexity exceed the capabilities of traditional relational database systems to capture, manage, and process within a reasonable time. Big Data is commonly characterized by the '5Vs':

- Volume: Extremely large amounts of data (gigabytes → terabytes → petabytes) that require distributed storage.
- Velocity: High speed of data generation and processing (real-time streams, near-real-time ingestion).
- Variety: Multiple data formats — structured (tables), semi-structured (JSON, XML), and unstructured (text, images, audio).
- Veracity: Data quality and trustworthiness — dealing with noisy, incomplete, or biased data.
- Value: Extracting actionable insights that generate business or societal value. Big Data vs Traditional Systems:
- Traditional RDBMS excel at transactional workloads and ACID guarantees but often struggle to scale horizontally for massive datasets.
- Big Data systems (Hadoop, Spark, NoSQL) trade off some relational features for scalability, partition tolerance, and high-throughput processing.

Big Data Lifecycle (high-level):

1. Data Generation (sensors, logs, social media, transactions)
2. Data Ingestion (Kafka, NiFi, Flume)
3. Storage (HDFS, object stores, NoSQL)
4. Processing (batch: MapReduce/Spark, streaming: Spark Structured Streaming, Flink)

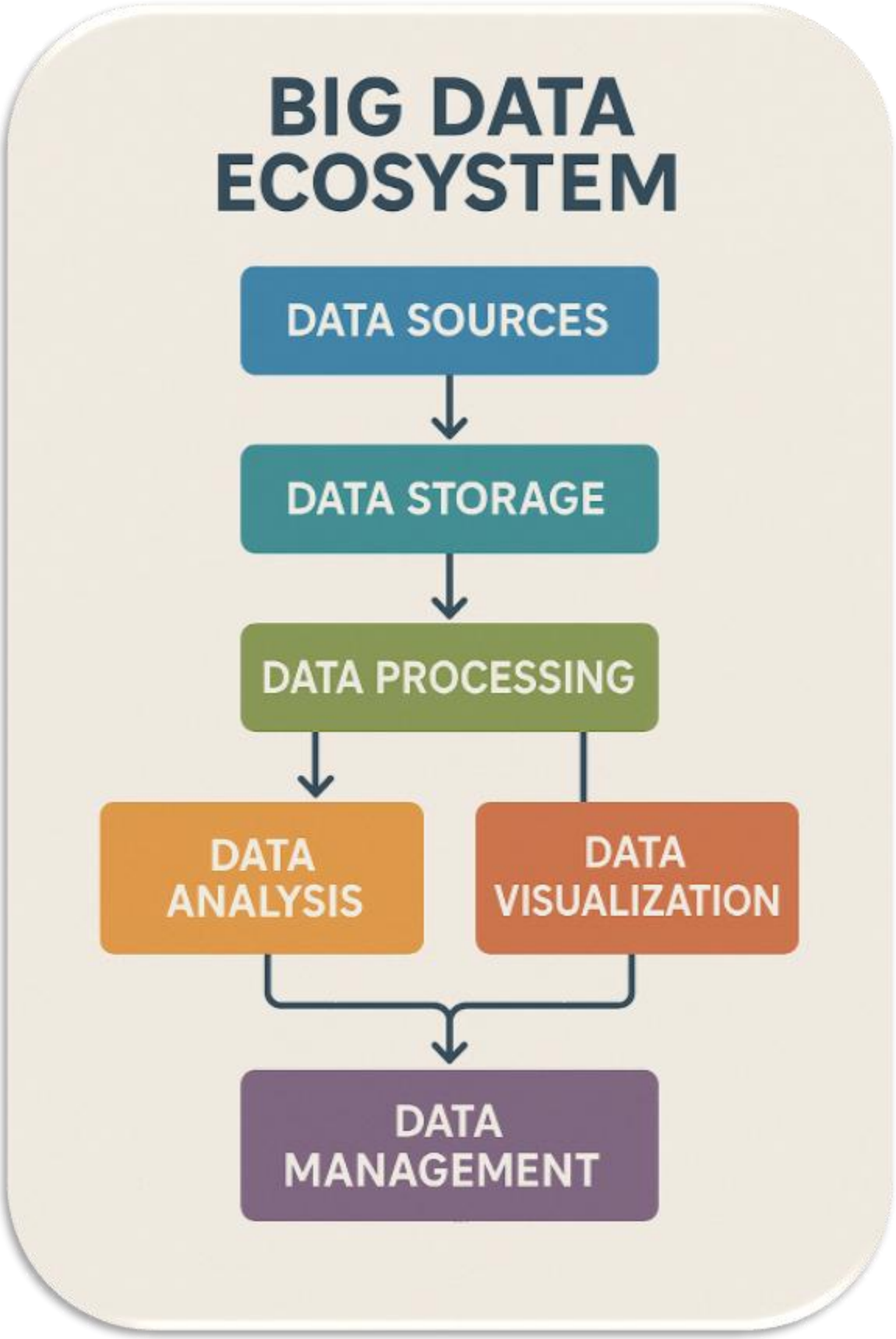
5. Analytics & ML (Spark MLlib, TensorFlow)
6. Visualization & Action (Tableau, Power BI, dashboards)

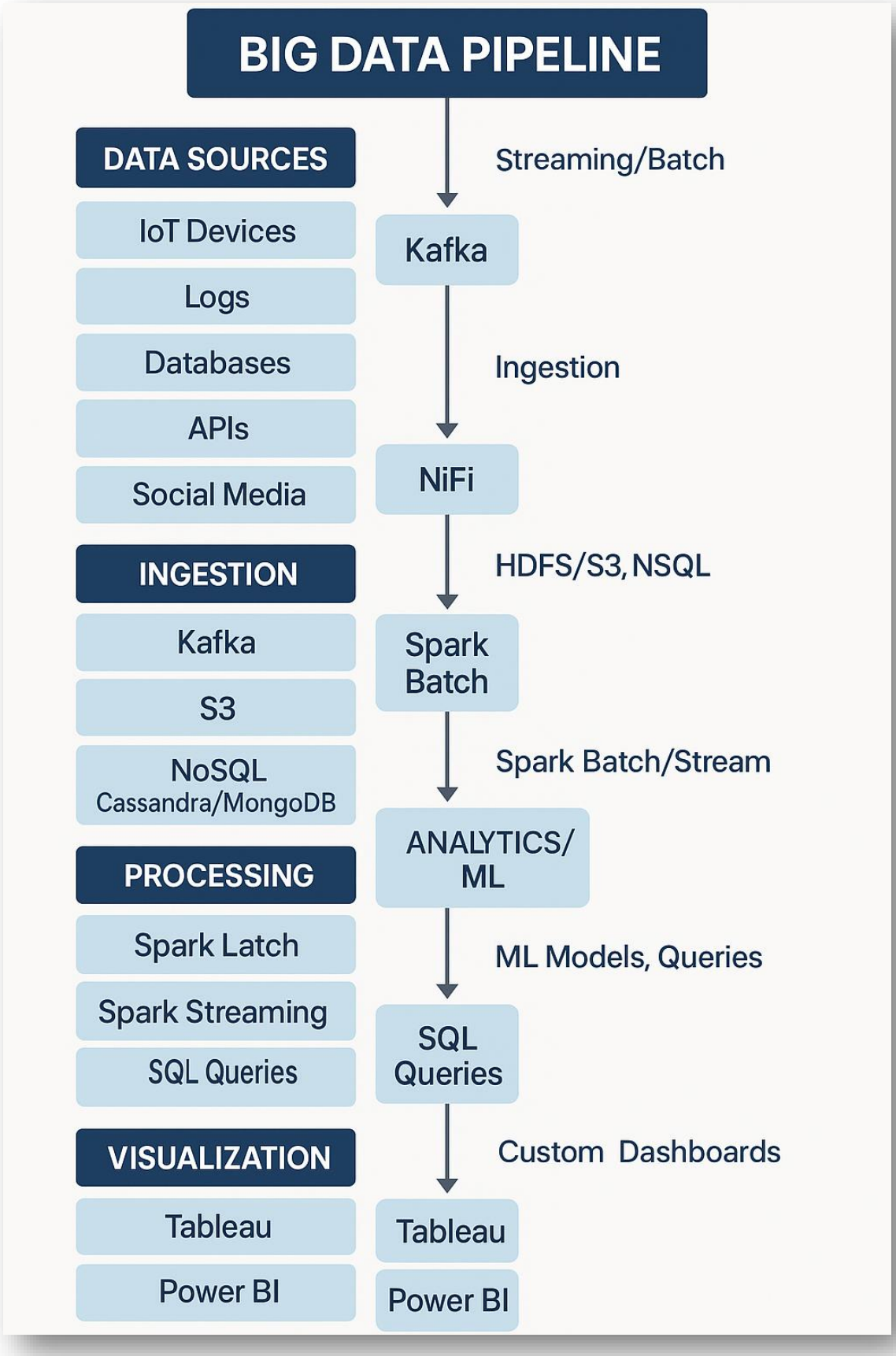
Architectural Patterns:

- Batch Processing: process large volumes of data in scheduled jobs (Hadoop MapReduce, Spark batch).
- Stream Processing: continuous processing of incoming events (Kafka + Spark Streaming / Structured Streaming).
- Lambda Architecture: combines batch and real-time layers to provide both accurate (batch) and low-latency (speed) views.
- Kappa Architecture: simplifies Lambda by using a single streaming pipeline for both real-time and reprocessing needs.

## Big Data Ecosystem




BIG DATA ECOSYSTEM
DATA SOURCES
DATA STORAGE
DATA PROCESSING
DATA ANALYSIS
DATA VISUALIZATION
DATA MANAGEMENT


BIG DATA PIPELINE
DATA SOURCES
IoT Devices
Logs
Databases
APIs
Social Media
INGESTION
Kafka
S3
NoSQL
Cassandra/MongoDB
PROCESSING
Spark Latch
Spark Streaming
SQL Queries
VISUALIZATION
Tableau
Power BI
Streaming/Batch
Kafka
Ingestion
NiFi
HDFS/S3, NSQL
Spark
Batch
Spark Batch/Stream
ANALYTICS/
ML
ML Models, Queries
SQL
Queries
Custom Dashboards
Tableau
Power BI

## Lab Exercise — Exploratory Analysis with pandas

Objective:
Perform exploratory data analysis on a moderately large dataset (World Bank WDI or a similar CSV) to observe Big Data characteristics and prepare the dataset for later processing with Spark.

Requirements:
- Python 3.8+ (Anaconda recommended)
- Jupyter Notebook or VS Code
- pandas, numpy, matplotlib, seaborn (pip install pandas numpy matplotlib seaborn)
- Optional: PySpark for larger-scale testing

Dataset:
Use World Bank WDI CSV or any multi-year country indicators dataset. Example link: https://databank.worldbank.org/source/world-development-indicators

Alternatively, use a large CSV (50MB+) to simulate scale for EDA.
Step-by-step Lab Tasks
1. Download the WDI CSV and place it in your working folder (e.g., data/WDI.csv).
2. Open a new Jupyter notebook and import pandas and numpy.
3. Load the CSV with pandas using chunking (if large) or read_csv normally for smaller files.
4. Compute basic dataset metrics: number of rows, columns, memory usage, missing values per column.
5. Sample rows (random and head/tail) to inspect variety and formatting issues.
6. Perform simple aggregations (group by country, year) and visualize trends for a chosen indicator.
7. Save a cleaned subset to a new CSV for later Spark processing.

Notes on chunking: For very large CSVs, use `pd.read_csv(..., chunksize=100000)` to iterate and compute increments to avoid memory errors.

## Sample Code — pandas (Exploratory - PySpark Word Count)

Use python script to load the **World Development Indicators (WDI)** dataset and also does some memory and missing-value checks, performs a yearly aggregation, and exports a small sample as follows:

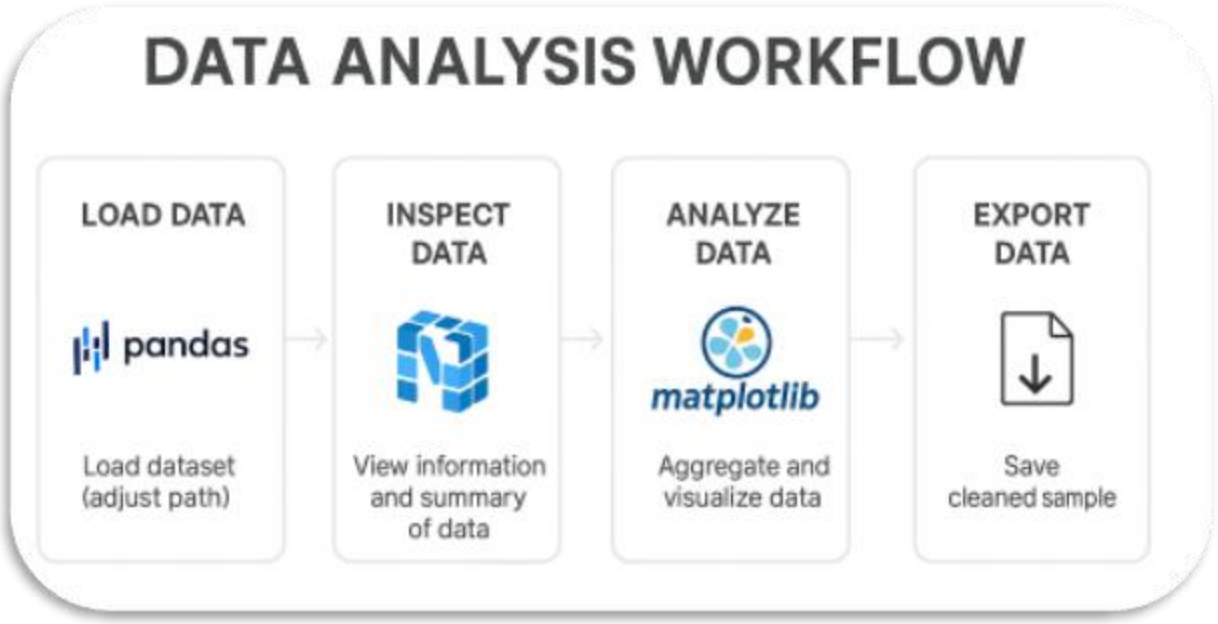
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load dataset (adjust path)
df = pd.read_csv('data/WDI.csv')
print('Rows:', len(df))
print('Columns:', df.shape[1])
print(df.info())

# Memory and missing values
print(df.memory_usage(deep=True).sum() / (1024**2), 'MB')
print(df.isnull().sum().sort_values(ascending=False).head(10))

# Simple aggregation example: average indicator by year
if 'Year' in df.columns and 'Value' in df.columns:
    yearly = df.groupby('Year')['Value'].mean().reset_index()
    plt.plot(yearly['Year'], yearly['Value'])
    plt.title('Average Indicator by Year')
    plt.show()

# Save cleaned sample
df_sample = df.sample(frac=0.01, random_state=42)  # 1% sample
df_sample.to_csv('data/WDI_sample.csv', index=False)
```



## Case Study

A Ministry of Health wants to analyse national electronic health records (EHRs) to monitor non-communicable diseases (diabetes, hypertension). The data comes from multiple hospitals, insurance claims, and mobile health apps.

Big Data Approach:

- Apply the 5Vs to characterize EHR data (Volume from millions of patient visits, Velocity from real-time monitoring devices, Variety across labs/images/text, Veracity with inconsistent entries, Value for policymaking).
- Students conduct exploratory analysis on a 50MB+ synthetic patient dataset using pandas.

Expected Outcomes:

Identify hotspots of chronic disease, reveal data quality gaps (e.g., missing HbA1c test results), and demonstrate how Big Data supports preventive health strategies.

## Suggested Readings

- Viktor Mayer-Schönberger & Kenneth Cukier, 'Big Data: A Revolution That Will Transform How We Live, Work, and Think' (Intro chapters).
- Nathan Marz, 'Big Data: Principles and Best Practices of Scalable Real-Time Data Systems' (selected chapters).
- Apache Hadoop and Spark official documentation (online).
- Selected tutorial articles on EDA with pandas.

# MODULE 2 – BIG DATA ECOSYSTEM AND ARCHITECTURES

## Learning Outcomes

By the end of this module, students should be able to:

- Describe the core components of the Big Data ecosystem (HDFS, YARN, MapReduce, Spark, Kafka, NoSQL).
- Explain batch, stream, and hybrid (Lambda, Kappa) architectures with their trade-offs.
- Identify when to use Hadoop, Spark, NoSQL, and streaming platforms in real-world scenarios.
- Understand how ecosystem components integrate into an end-to-end pipeline.
- Compare cloud-native vs on-premises Big Data architectures.

## Theory Overview

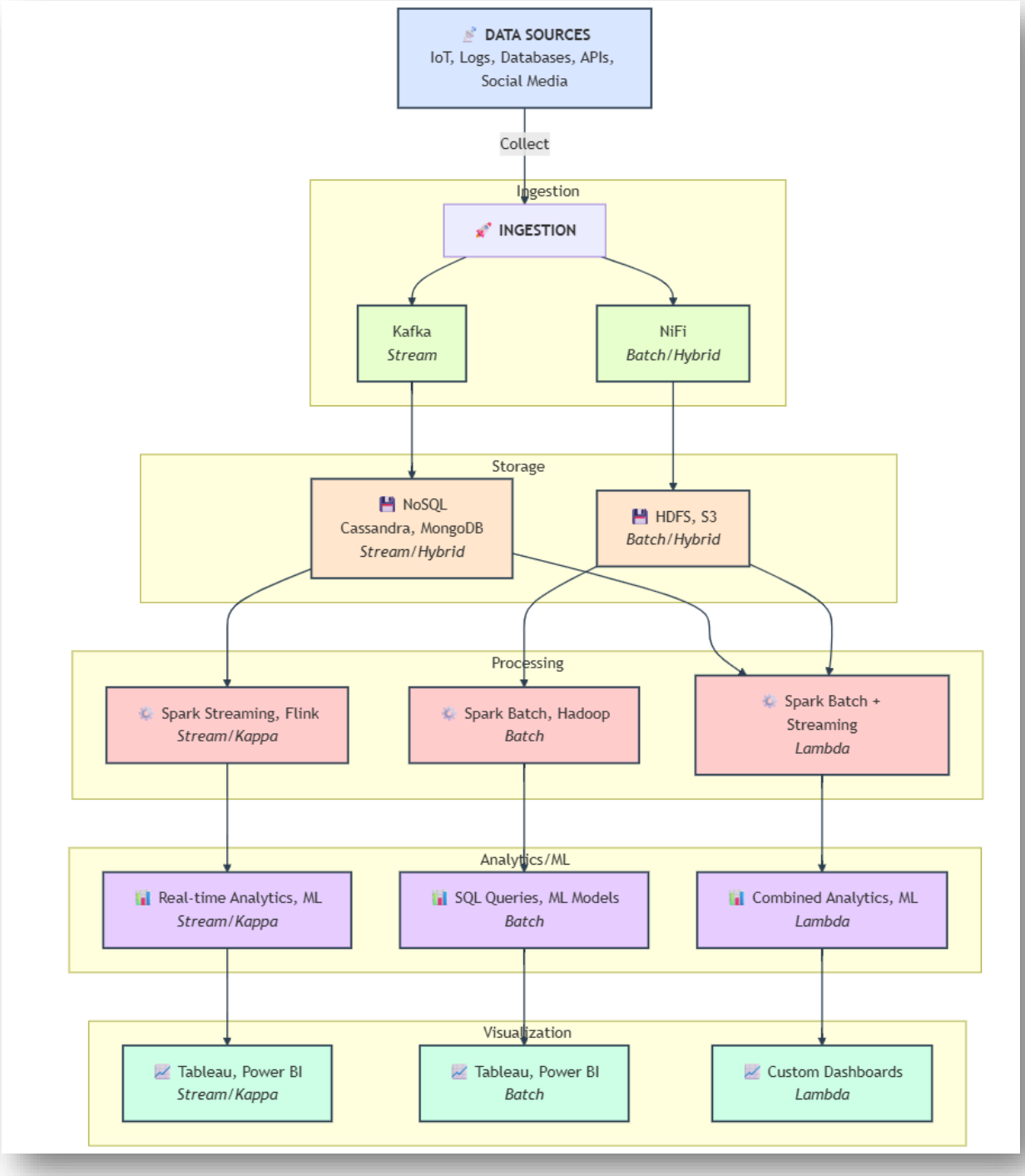
The Big Data ecosystem is composed of multiple technologies that address storage, processing, ingestion, and analysis of large-scale datasets. The ecosystem evolves rapidly, but several foundational components remain central:

- **HDFS (Hadoop Distributed File System):** Distributed, fault-tolerant storage optimized for large files and batch processing.
- **YARN (Yet Another Resource Negotiator):** Cluster resource manager that schedules tasks across nodes in the Hadoop ecosystem.
- **MapReduce:** Programming model for distributed batch processing; foundation of early Hadoop systems.
- **Apache Spark:** Unified processing engine for batch and streaming with APIs in Scala, Python, Java, R. Faster than MapReduce.
- **Apache Kafka:** Distributed publish-subscribe messaging system used for high-throughput real-time data ingestion.
- **NoSQL Databases:** Schema-less databases (MongoDB, Cassandra, HBase) designed for scalability and handling diverse data types.

Architectural Patterns

- **Batch-Oriented:** Suited for ETL and periodic analytics (Hadoop MapReduce, Spark batch).
- **Stream-Oriented:** Designed for real-time event processing (Kafka, Spark Streaming, Flink).
- **Lambda Architecture:** Combines batch and stream layers for both accuracy and speed.
- **Kappa Architecture:** Simplifies by using a single streaming pipeline, avoiding dual maintenance.

# Big Data Reference Architectures



## Lab Exercise — Setting up Hadoop and Spark

Objective:

Gain hands-on familiarity with the Hadoop and Spark ecosystem.

- **Option A:** Install Hadoop and Spark locally (pseudo-distributed mode).
- **Option B:** Use cloud-based environments (Google Colab for PySpark, or AWS/GCP free tiers).

Requirements

- Java 8+ (for Hadoop and Spark local install)
- Hadoop 3.x (download from Apache mirrors)
- Spark 3.x (pre-built with Hadoop binaries)
- Python 3.8+ with pyspark package (pip install pyspark)
- Cloud option: Google Colab (pre-configured PySpark runtime)

Step-by-Step Tasks

1. Install Java and configure JAVA_HOME environment variable.
2. Download Hadoop, extract, and configure core-site.xml and hdfs-site.xml for pseudo-distributed mode.
3. Format HDFS namenode and start dfs & yarn daemons.
4. Download Spark pre-built package, configure environment variables (SPARK_HOME, PATH).
5. Start Spark shell (Scala or PySpark).
6. Verify HDFS by creating directories and uploading small files.
7. Run a sample Spark job (word count) to test integration.

## Sample Code — PySpark Word Count (phyton)

```
from pyspark import SparkContext
sc = SparkContext(appName='WordCount')

# Load text file
text_rdd = sc.textFile('hdfs:///user/data/sample.txt')

# Split into words and count
word_counts = text_rdd.flatMap(lambda line: line.split())    .map(lambda word: (word, 1))    .reduceByKey(lambda a, b: a + b)

# Collect results
for word, count in word_counts.take(10):
    print(word, count)

sc.stop()
```

## Case Study

A university hospital develops a hybrid architecture to support both retrospective research (batch analytics of medical histories) and real-time alerts (streaming from ICU monitors).

Big Data Approach:

- Batch layer: Hadoop for long-term storage of EHR and medical imaging.
- Speed layer: Spark + Kafka for real-time vitals monitoring (heart rate, oxygen saturation).
- Serving layer: APIs feeding dashboards for clinicians.

Expected Outcomes:

Clinicians receive immediate alerts for critical vitals (e.g., early warning score for sepsis), while researchers access long-term trends for clinical studies and AI model training.

## Suggested Readings


- Tom White, *Hadoop: The Definitive Guide*, O'Reilly Media.
- Matei Zaharia et al., *Learning Spark*, O'Reilly Media.
- Kafka official documentation: https://kafka.apache.org
- Dean & Ghemawat, *MapReduce: Simplified Data Processing on Large Clusters* (Google Research Paper).

# MODULE 3 – DATA STORAGE AND MANAGEMENT

## Learning Outcomes

By the end of this module, students should be able to:

- Differentiate between traditional RDBMS, NoSQL, and distributed file systems.
- Explain CAP theorem and its implications for database design.
- Describe HDFS internals: blocks, replication, NameNode, DataNodes.
- Work with basic HDFS commands (upload, list, copy, delete).
- Compare columnar vs row-based storage for analytics workloads.
- Understand cloud-based storage options (Amazon S3, Google Cloud Storage, Azure Blob).

## Theory Overview

Data Storage Challenges in Big Data

- **Scalability:** Petabyte-scale datasets exceed the limits of single machines.
- **Variety:** Storage must handle structured, semi-structured, and unstructured data.
- **Fault Tolerance:** Systems must tolerate node failures without losing data.
- **Latency Requirements:** Some applications demand real-time access; others can tolerate batch access.

Distributed File Systems

- **HDFS (Hadoop Distributed File System):** Splits files into large blocks (default 128MB/256MB) stored across DataNodes. Provides replication (default 3 copies) for fault tolerance. A NameNode manages metadata.
- **GFS (Google File System):** Inspiration for HDFS, built by Google for internal large-scale data processing.

CAP Theorem (Consistency, Availability, Partition Tolerance)

- **Consistency:** Every read receives the most recent write.
- **Availability:** Every request receives a response (even if not the latest data).
- **Partition Tolerance:** System continues to operate despite network partitions. In distributed systems, we can only achieve two of these three guarantees simultaneously.

Databases for Big Data

- **Relational (RDBMS):** Good for structured data, but limited scaling.
- **NoSQL:**
    - *Key-Value Stores* (Redis, DynamoDB) – fast lookups.
    - *Document Stores* (MongoDB, CouchDB) – flexible JSON-based schemas.
    - *Columnar Stores* (HBase, Cassandra) – efficient for analytical queries.

- *Graph Databases* (Neo4j) – optimized for relationship-heavy queries.

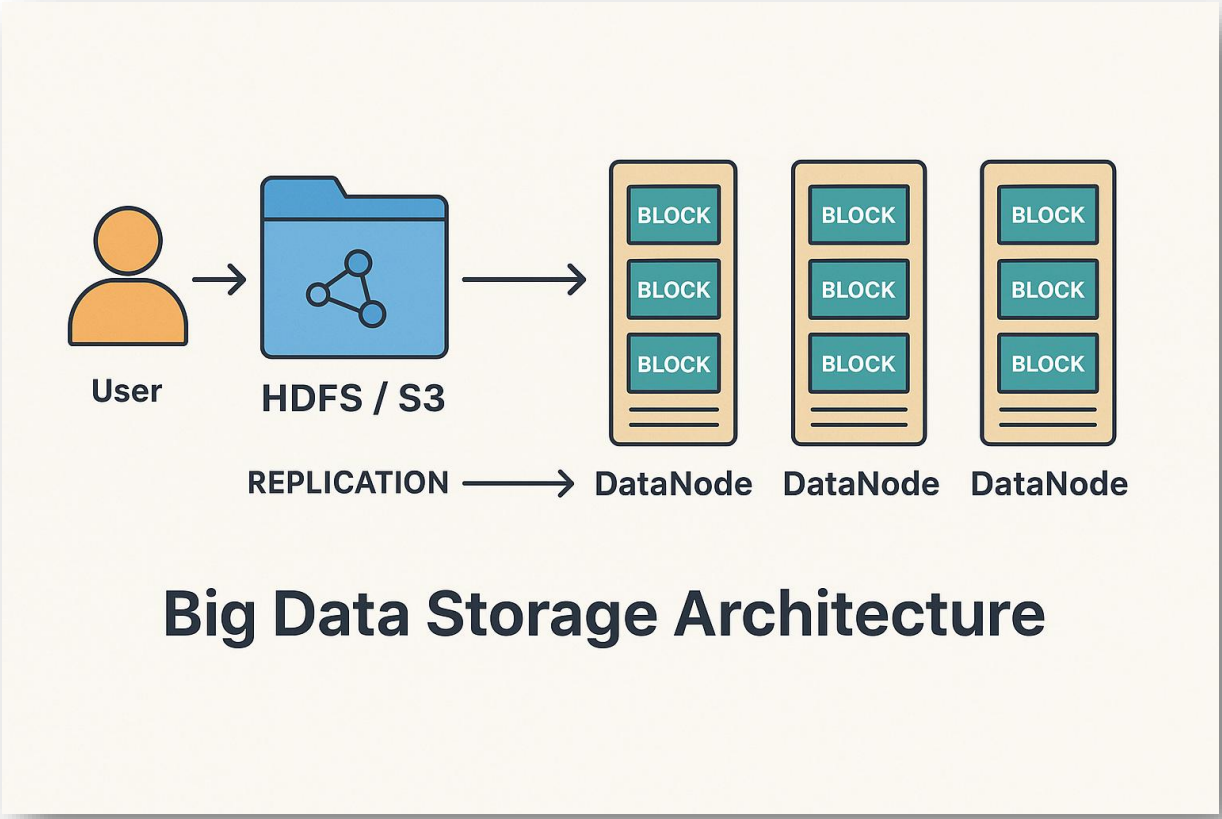
Row vs Columnar Storage

- **Row-Oriented:** Efficient for transactional systems (OLTP).
- **Column-Oriented:** Efficient for analytical queries (OLAP), because queries often scan only certain columns. Examples: Parquet, ORC.

Cloud Storage

- Amazon S3, Google Cloud Storage, Azure Blob Storage: object-based storage with high availability, integration with analytics platforms.

## Big Data Storage Architecture



## Lab Exercise — Working with HDFS

Objective:
Explore HDFS commands and understand distributed storage.

Requirements:
Hadoop installed (pseudo-distributed mode) OR access to a Hadoop cluster.
Step-by-Step Tasks

```
# 1. Start HDFS services
start-dfs.sh

# 2. Create a new HDFS directory
hdfs dfs -mkdir /user/student

# 3. Upload a local file to HDFS
hdfs dfs -put localfile.txt /user/student/

# 4. List files in HDFS
hdfs dfs -ls /user/student/

# 5. Display file contents
hdfs dfs -cat /user/student/localfile.txt

# 6. Replication test: check blocks & locations
hdfs fsck /user/student/localfile.txt -files -blocks -locations
```

## Sample Code — PySpark with Parquet (phyton):

```
from pyspark.sql import SparkSession

# Initialize Spark session
spark = SparkSession.builder.appName("StorageExample").getOrCreate()

# Load CSV
df = spark.read.csv("hdfs:///user/student/data.csv", header=True, inferSchema=True)

# Save as Parquet (columnar storage)
df.write.mode("overwrite").parquet("hdfs:///user/student/data_parquet")

# Read back Parquet file
df_parquet = spark.read.parquet("hdfs:///user/student/data_parquet")
df_parquet.show(5)

spark.stop()
```

## Case Study

A genomics lab sequences thousands of patient genomes daily (terabytes of data). Data must be stored securely, shared with researchers, and queried efficiently.

Big Data Approach:

- Students explore HDFS for distributed storage.
- Compare row-based (CSV) vs. columnar (Parquet) for genomic queries.
- Apply CAP theorem trade-offs in choosing Cassandra for availability and partition tolerance.

Expected Outcomes:

Efficient storage reduces costs, speeds genomic queries (mutation frequency analysis), and ensures scalable precision medicine pipelines.

## Suggested Readings

- Tom White, *Hadoop: The Definitive Guide* (Chapters on HDFS).
- MongoDB, Cassandra official documentation.
- Apache Parquet documentation.
- Brewer's CAP Theorem original paper (2000).

# MODULE 4 – HADOOP ECOSYSTEM

## Learning Outcomes

By the end of this module, students should be able to:

- Explain the major components of the Hadoop ecosystem.
- Differentiate between Hadoop's storage (HDFS), resource management (YARN), and processing (MapReduce, Hive, Pig, etc.).
- Run basic MapReduce jobs.
- Understand how Hive and Pig simplify working with large datasets.
- Integrate Hadoop ecosystem tools into end-to-end workflows.

## Theory Overview

The Hadoop ecosystem consists of multiple open-source projects designed to support large-scale data storage and processing.

Core Components

- **HDFS (Hadoop Distributed File System):** Distributed, fault-tolerant storage.
- **YARN (Yet Another Resource Negotiator):** Manages resources and schedules jobs across the cluster.
- **MapReduce:** Programming model for large-scale batch processing.

Data Processing & Querying Tools

- **Apache Hive:** Data warehouse tool; allows SQL-like queries (HiveQL) on HDFS data.
- **Apache Pig:** High-level dataflow language (Pig Latin) for ETL and transformations.
- **Apache HBase:** Column-oriented NoSQL database built on top of HDFS.

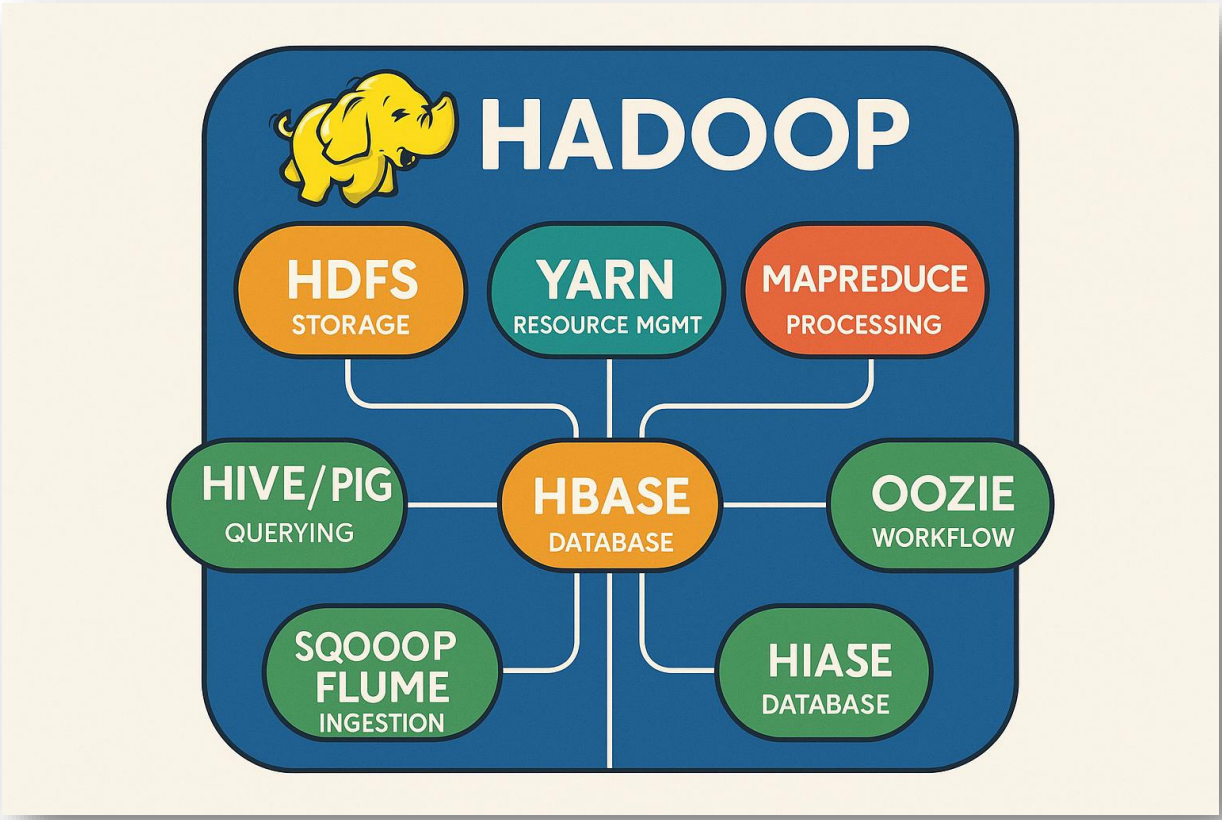
Data Ingestion & Integration

- **Apache Sqoop:** Transfers data between relational databases (MySQL, PostgreSQL, Oracle) and Hadoop.
- **Apache Flume:** Collects, aggregates, and moves large amounts of log data into HDFS.

Workflow & Orchestration

- **Apache Oozie:** Job scheduling and orchestration for Hadoop workflows.

## Hadoop Ecosystem Components




## Lab Exercise — Running Hadoop and Hive

Objective:
Run a simple MapReduce word count job and query data using Hive.

Requirements

- Hadoop (pseudo-distributed or cluster mode).
- Hive installed and configured.

Step-by-Step Tasks

```
# 1. Place input text into HDFS (bash)
hdfs dfs -put sample.txt /user/student/input/

# 2. Run built-in MapReduce word count example (bash)
hadoop jar /usr/local/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar wordcount /user/student/input /user/student/output

# 3. View results (bash)
hdfs dfs -cat /user/student/output/part-r-00000 | head

# 4. Create Hive table from a CSV file in HDFS (sql)
CREATE TABLE students (
    id INT,
    name STRING,
    gpa FLOAT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

# 5. Load data and run a query (sql)
LOAD DATA INPATH '/user/student/students.csv' INTO TABLE students;
SELECT name, gpa FROM students WHERE gpa > 3.0;
```

## Sample Code — MapReduce in Python (Hadoop Streaming)

```
# mapper.py
#!/usr/bin/env python3
import sys
for line in sys.stdin:
    for word in line.strip().split():
        print(f"{word}\t1")

# reducer.py
#!/usr/bin/env python3
import sys
from collections import defaultdict

counts = defaultdict(int)
for line in sys.stdin:
    word, num = line.strip().split("\t")
    counts[word] += int(num)

for word, count in counts.items():
    print(f"{word}\t{count}")

# Run with Hadoop Streaming (bash)
hadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-*.jar   -input /user/student/input   -output /user/student/output2   -mapper mapper.py   -reducer reducer.py
```

## Case Study:

A national immunization registry integrates childhood vaccination data across provinces. Researchers need insights into coverage rates and dropout patterns.

Big Data Approach:

- Use Hive to run SQL queries over millions of vaccination records.
- Apply Pig scripts for ETL (extract birth cohorts, filter missed doses).
- Integrate with HBase for quick lookups on individual immunization histories.

Expected Outcomes:

Detect under-vaccinated districts, predict outbreaks of preventable diseases, and inform public health interventions.

## Suggested Readings


- Tom White, *Hadoop: The Definitive Guide*.
- Alan Gates, *Programming Pig*.
- Dean Wampler et al., *Programming Hive*.
- Apache HBase official documentation.

# MODULE 5 – MAPREDUCE PROGRAMMING

## Learning Outcomes

By the end of this module, students should be able to:

- Explain the MapReduce programming model (Map, Shuffle & Sort, Reduce).
- Develop basic MapReduce programs in Java and Python (Hadoop Streaming).
- Understand the flow of key-value pairs through the stages.
- Optimize MapReduce jobs using combiners and partitioners.
- Apply MapReduce to solve word count, log analysis, and join problems.

## Theory Overview

MapReduce Model

- **Map Function:** Processes input splits and emits intermediate key-value pairs.
- **Shuffle & Sort:** Groups values by keys across nodes.
- **Reduce Function:** Aggregates grouped values and produces final output.

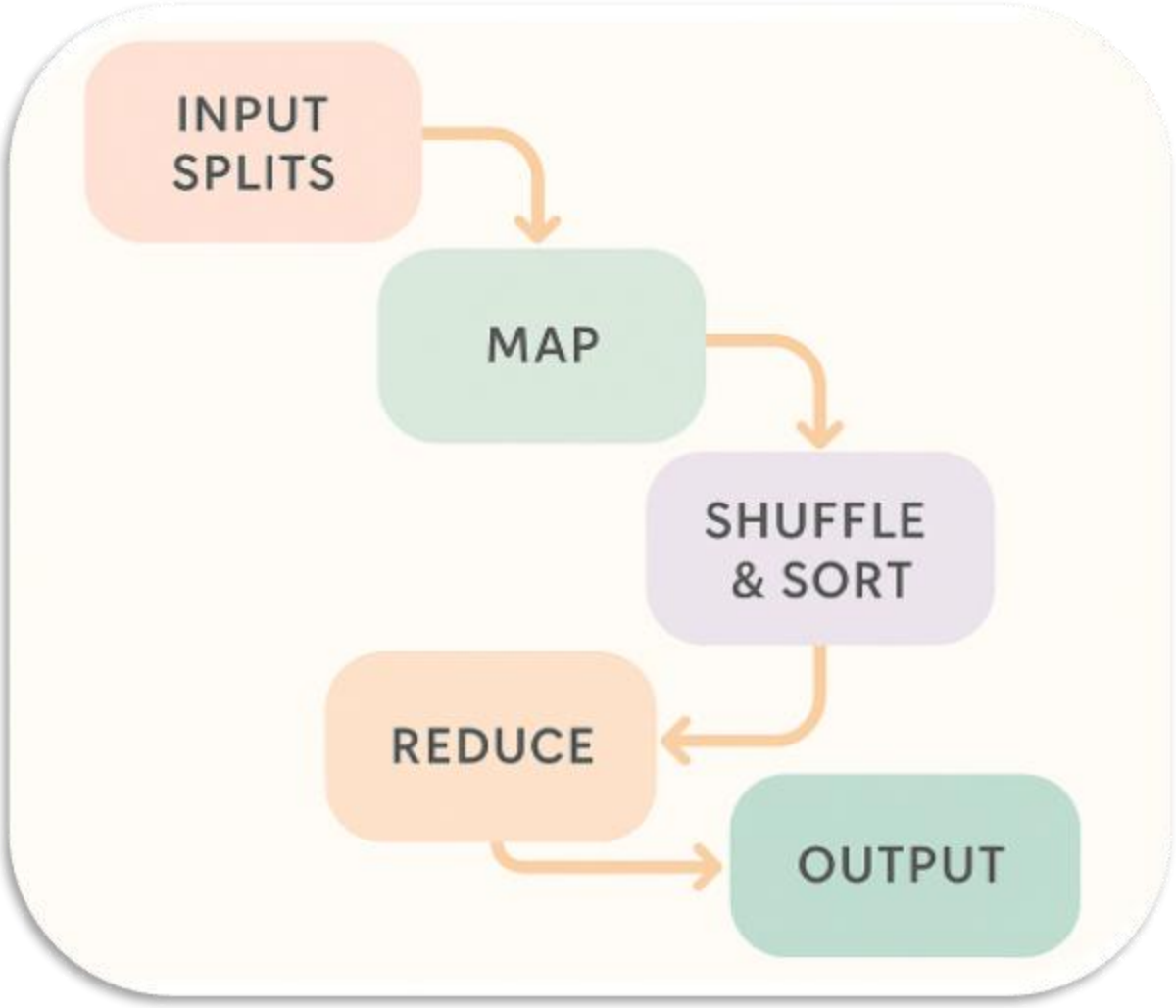
Execution Flow

1. Input data is split into chunks.
2. Each chunk is processed by a Map task → emits key-value pairs.
3. Keys are shuffled and sorted across nodes.
4. Reduce tasks aggregate results.
5. Output is written to HDFS.

Key Concepts

- **Combiners:** Local reducers that minimize data transfer.
- **Partitioners:** Decide which reducer processes a given key.
- **Fault Tolerance:** Failed tasks are automatically rescheduled.
- **Data Locality:** Hadoop tries to run tasks close to where the data resides.

## MapReduce Data Flow



## Lab Exercise — Word Count in Java

Objective:

Implement the classic word count program in Java using MapReduce API.

Java Code Example

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {
  public static class TokenizerMapper
       extends Mapper<Object, Text, Text, IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {
      String[] tokens = value.toString().split("\\s+");
      for (String token : tokens) {
        word.set(token);
        context.write(word, one);
      }
    }
  }

  public static class IntSumReducer
       extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values,
                       Context context)
        throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable val : values) {
        sum += val.get();
      }
      context.write(key, new IntWritable(sum));
    }
  }

  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

## Sample Code — Hadoop Streaming with Python

# mapper.py

```
#!/usr/bin/env python3
import sys
for line in sys.stdin:
    for word in line.strip().split():
        print(f"{word}\t1")
```

# reducer.py

```
#!/usr/bin/env python3
import sys
from collections import defaultdict

counts = defaultdict(int)
for line in sys.stdin:
    word, num = line.strip().split("\t")
    counts[word] += int(num)

for word, count in counts.items():
    print(f"{word}\t{count}")
```

Run with Hadoop Streaming:

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
  -input /user/student/input \
  -output /user/student/output3 \
  -mapper mapper.py \
  -reducer reducer.py
```

## Case Study

A global health NGO analyses log files from contact tracing apps during an epidemic. The dataset has billions of daily proximity interactions.

Big Data Approach:

- Students implement MapReduce jobs to count high-risk contacts by region.
- Use combiners to optimize efficiency and reduce data transfer.

Expected Outcomes:

Generate heatmaps of disease transmission networks, enabling authorities to deploy targeted testing and quarantine.

## Suggested Readings

- Tom White, *Hadoop: The Definitive Guide* (Chapter on MapReduce).
- Lin & Dyer, *Data-Intensive Text Processing with MapReduce*.
- Apache Hadoop Streaming documentation.

# MODULE 6 – APACHE SPARK BASICS

## Learning Outcomes

By the end of this module, students should be able to:

- Explain the core architecture of Apache Spark (driver, executors, cluster manager).
- Differentiate Spark from Hadoop MapReduce in terms of performance and usability.
- Work with Spark RDDs and DataFrames.
- Run basic Spark jobs using PySpark.
- Understand Spark execution flow (lazy evaluation, DAG scheduler).

## Theory Overview

What is Apache Spark?

Apache Spark is a distributed data processing framework designed for speed and ease of use. It improves upon Hadoop MapReduce by using in-memory computation, fault tolerance, and unified APIs for batch, streaming, SQL, and machine learning.

Spark Components

- **Driver Program:** Runs the main function and coordinates execution.
- **Cluster Manager:** Allocates resources (YARN, Mesos, Kubernetes, or standalone).
- **Executors:** Worker processes that run tasks.

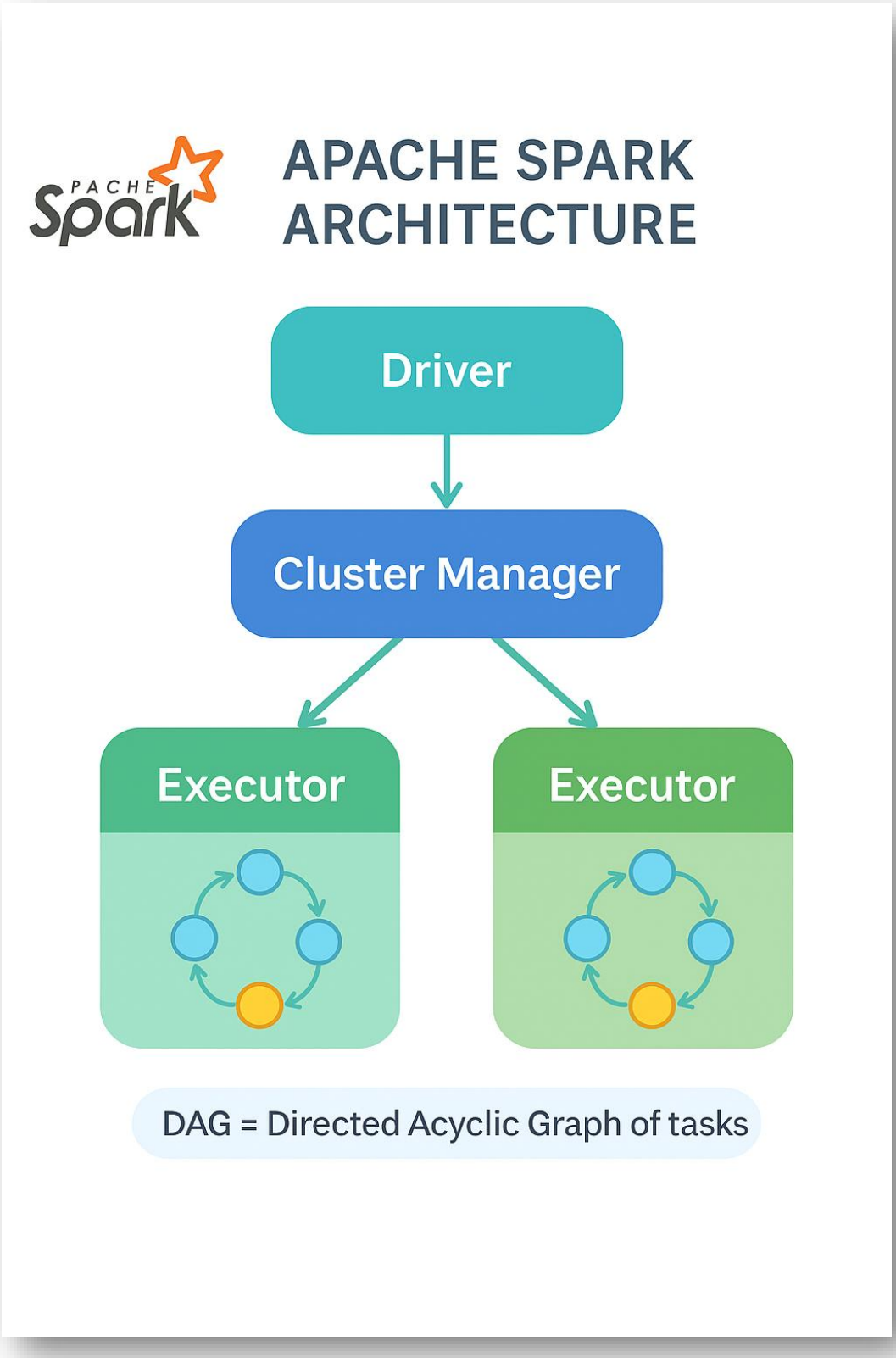
Spark vs Hadoop MapReduce

- **In-memory processing** → faster than disk-based MapReduce.
- **Unified APIs** → Spark supports batch, streaming, SQL, and ML in one framework.
- **Ease of use** → APIs available in Python, Scala, Java, R.

Core Abstractions

- **RDD (Resilient Distributed Dataset):** Immutable, fault-tolerant collection of objects distributed across nodes. Supports transformations (map, filter) and actions (collect, count).
- **DataFrames:** Higher-level abstraction with schema; optimized by Catalyst query optimizer.
- **Lazy Evaluation:** Transformations build a logical DAG; execution occurs only when an action is triggered.

## Spark Architecture



## Lab Exercise — Getting Started with PySpark

Objective:
Write and execute simple Spark programs.

Step-by-Step Tasks

```
# 1. Install PySpark
pip install pyspark

# 2. Start Spark shell
pyspark

# 3. Create an RDD from a list
data = [1, 2, 3, 4, 5]
rdd = sc.parallelize(data)
print("Sum:", rdd.reduce(lambda a, b: a + b))

# 4. Create a DataFrame from a CSV file
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Basics").getOrCreate()

df = spark.read.csv("data.csv", header=True, inferSchema=True)
df.show(5)

# 5. Run a SQL query
df.createOrReplaceTempView("people")
result = spark.sql("SELECT name, age FROM people WHERE age > 30")
result.show()
```

## Sample Code — Word Count with RDD

```
from pyspark import SparkContext

sc = SparkContext(appName="WordCountRDD")

text_rdd = sc.textFile("hdfs:///user/student/sample.txt")
word_counts = (text_rdd
               .flatMap(lambda line: line.split())
               .map(lambda word: (word, 1))
               .reduceByKey(lambda a, b: a + b))

print(word_counts.take(10))

sc.stop()
```

## Case Study

A cancer research institute needs real-time analysis of pathology images and biopsy reports for diagnosis assistance.

Big Data Approach:
Students use RDDs and DataFrames to process unstructured text (pathology notes) and structured data (patient records).
Compare Spark vs. MapReduce for speed in analysing biopsy metadata.

Expected Outcomes:
Faster detection of suspicious pathology patterns supports early cancer diagnosis and reduces clinician workload.

## Suggested Readings

- Matei Zaharia et al., *Learning Spark* (O'Reilly).
- Apache Spark documentation.
- Karau et al., *High Performance Spark*.

# MODULE 7 – SPARK SQL AND DATAFRAMES

## Learning Outcomes

By the end of this module, students should be able to:

- Explain the role of Spark SQL in the Spark ecosystem.
- Differentiate between RDDs and DataFrames.
- Load, query, and transform structured data using DataFrames.
- Use SQL queries within Spark programs.
- Optimize queries using Catalyst and Tungsten.

## Theory Overview

What is Spark SQL?

Spark SQL is the Spark module for processing structured data. It provides:

- A **DataFrame API** similar to Pandas or R.
- A **SQL interface** for querying structured data.
- Optimizations via the **Catalyst query optimizer** and **Tungsten execution engine**.

DataFrames vs RDDs

- **RDDs:** Low-level, untyped distributed collections.
- **DataFrames:** High-level, tabular representation with schema, enabling optimizations.
- **Dataset API (Scala/Java only):** Strongly typed extension of DataFrames.

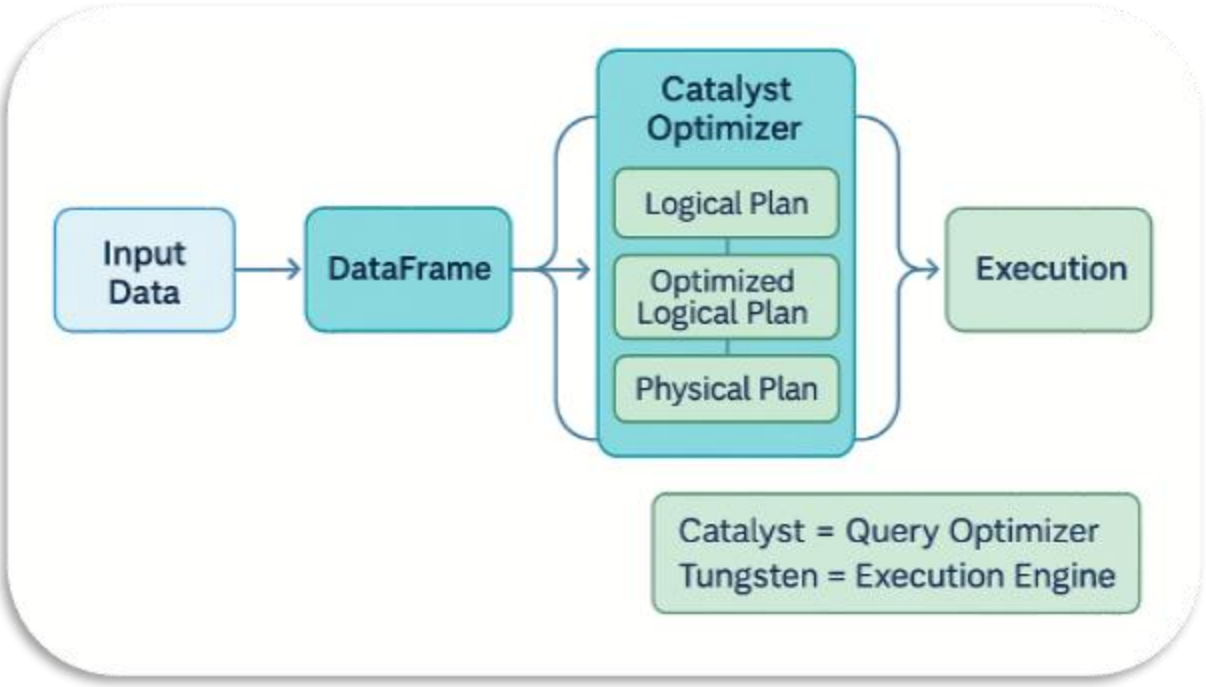
Optimizations

- **Catalyst Optimizer:** Rewrites queries for efficiency (predicate pushdown, column pruning).
- **Tungsten Engine:** Optimizes memory and CPU usage with whole-stage code generation.

Supported Data Sources

- CSV, JSON, Parquet, ORC, Avro.
- JDBC connections to RDBMS.
- Hive integration for SQL queries.

## Spark SQL Workflow



## Lab Exercise — Spark SQL with DataFrames

Objective:
Work with DataFrames and SQL queries in Spark.

Step-by-Step Tasks

Load a CSV file into a DataFrame:

```
from pyspark.sql import SparkSession

# 1. Initialize SparkSession
spark = SparkSession.builder.appName("SparkSQLDemo").getOrCreate()

# 2. Load CSV into DataFrame
df = spark.read.csv("employees.csv", header=True, inferSchema=True)
df.printSchema()
df.show(5)

# 3. Run basic DataFrame transformations
df_filtered = df.filter(df.salary > 50000)
df_grouped = df.groupBy("department").avg("salary")
df_grouped.show()

# 4. Register a temporary SQL view and run query
df.createOrReplaceTempView("employees")
result = spark.sql("""
    SELECT department, AVG(salary)
    FROM employees
    GROUP BY department
""")
result.show()

# 5. Save results as Parquet
result.write.mode("overwrite").parquet("output/avg_salary_by_dept")
```

## Sample Code — JSON + SQL Integration

```
# 1. Load JSON file into a DataFrame
json_df = spark.read.json("people.json")
json_df.show()

# 2. Register DataFrame as a temporary SQL view
json_df.createOrReplaceTempView("people")

# 3. Query with SQL
adults = spark.sql(
    """
    SELECT name, age
    FROM people
    WHERE age >= 18
    """
)
adults.show()
```

## Case Study

A hospital group merges multiple EHR systems after acquisitions. To analyse patient flows, data must be integrated across structured (SQL tables) and semi-structured (HL7, FHIR JSON) formats.

Big Data Approach:

- Load patient admission records into Spark DataFrames.
- Use Spark SQL to query trends (ER admissions by day, readmission rates).
- Compare schema evolution handling in JSON vs. relational formats.

Expected Outcomes:

Standardized insights across hospital networks improve operational efficiency and patient safety monitoring.

## Suggested Readings

- Matei Zaharia et al., *Learning Spark* (Chapters on Spark SQL).
- Apache Spark SQL documentation.
- Karau et al., *High Performance Spark*.

# MODULE 8 – SPARK STREAMING

## Learning Outcomes

By the end of this module, students should be able to:

- Explain the concept of real-time data processing.
- Describe Spark Streaming's micro-batch architecture.
- Build a streaming pipeline using Spark Structured Streaming.
- Integrate streaming data sources (e.g., socket, Kafka, files).
- Apply transformations and aggregations on streaming data.

## Theory Overview

Real-Time Data Processing

- **Batch processing:** Works on large, static datasets.
- **Streaming processing:** Processes data in motion, often with low latency.
- **Use cases:** Fraud detection, social media monitoring, IoT sensor data, stock market feeds.

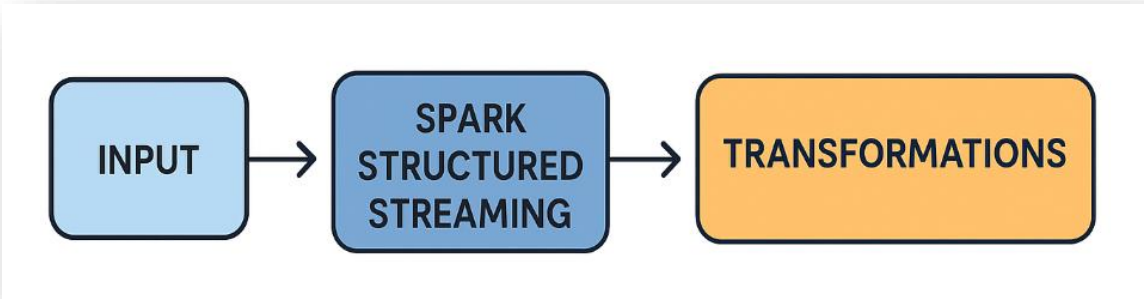
Spark Streaming Architecture

- **DStreams (legacy):** Discretized streams split into small RDD batches.
- **Structured Streaming (recommended):** Built on DataFrames and SQL engine, uses continuous processing or micro-batching.

Key Concepts

- **Input sources:** Kafka, socket, files, cloud storage, etc.
- **Transformations:** map, filter, groupBy, window operations.
- **Output sinks:** Console, memory, files, Kafka, JDBC.
- **Fault tolerance:** Checkpointing and write-ahead logs.

## Spark Structured Streaming Workflow



## Lab Exercise — Streaming Word Count

Objective:
Create a real-time word count from a network socket stream.

Step-by-Step Tasks

### 1. Start a socket server in a terminal (bash)

```
nc -lk 9999
```

### 2. Spark Structured Streaming code (Python)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("StreamingWordCount").getOrCreate()

# Read data stream from socket
lines = spark.readStream.format("socket") \    .option("host", "localhost") \    .option("port", 9999) \    .load()

# Split into words
words = lines.selectExpr("explode(split(value, ' ')) as word")

# Count words
word_counts = words.groupBy("word").count()

# Output to console
query = word_counts.writeStream \    .outputMode("complete") \    .format("console") \    .start()

query.awaitTermination()
```

### 3. Type text into the socket terminal and see live counts appear in Spark

```
Input:
hello hello world

Output:
+-----+-----+
| word|count|
+-----+-----+
|hello|    2|
|world|    1|
|spark|    1|
+-----+-----+
```

### 1. Start a socket server in a terminal (bash)

```
nc -lk 9999
```

### 2. Spark Structured Streaming code (Python)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("StreamingWordCount").getOrCreate()

# Read data stream from socket
lines = spark.readStream.format("socket") \    .option("host", "localhost") \    .option("port", 9999) \    .load()

# Split into words
words = lines.selectExpr("explode(split(value, ' ')) as word")

# Count words
word_counts = words.groupBy("word").count()

# Output to console
query = word_counts.writeStream \    .outputMode("complete") \    .format("console") \    .start()

query.awaitTermination()
```

### 3. Type text into the socket terminal and see live counts appea

```
Input:
hello hello world

Output:
+-----+-----+
| word|count|
+-----+-----+
|hello|    2|
|world|    1|
|spark|    1|
+-----+-----+
```

### 4. Kafka Streaming Example (Python)

```
# Kafka streaming example
df = spark.readStream.format("kafka") \    .option("kafka.bootstrap.servers", "localhost:9092") \    .option("subscribe", "test-topic") \    .load()

df.selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)").writeStream \    .outputMode("append") \    .format("console") \    .start() \    .awaitTermination()
```

## Case Study

An ICU deploys wearable devices to continuously stream vitals (ECG, $SpO_2$, blood pressure) for critically ill patients.

Big Data Approach:

- Students build Spark Structured Streaming pipelines with Kafka.
- Implement transformations to detect abnormal patterns (tachycardia, oxygen desaturation).
- Use checkpointing for resilience.

Expected Outcomes:

Real-time dashboards notify clinicians of early signs of deterioration, reducing mortality rates from delayed interventions.

## Suggested Readings

- Apache Spark Structured Streaming Documentation.
- Databricks Guide to Structured Streaming.
- Karau et al., *High Performance Spark*.

# MODULE 9 – NOSQL DATABASES FOR BIG DATA

## Learning Outcomes

By the end of this module, students should be able to:

- Explain the limitations of relational databases for big data workloads.
- Differentiate between types of NoSQL databases (key-value, document, column, graph).
- Demonstrate CRUD operations on at least one NoSQL system.
- Understand consistency, availability, and partition tolerance (CAP theorem).
- Integrate NoSQL databases with big data frameworks like Spark.

## Theory Overview

Why NoSQL?

- Relational databases struggle with scalability, schema flexibility, and high-velocity data.
- NoSQL databases offer horizontal scaling, flexible schemas, and optimized data models.

Types of NoSQL Databases

1. **Key-Value Stores** – e.g., Redis, DynamoDB.
    - Store data as key-value pairs.
    - Best for caching, session storage, fast lookups.
2. **Document Stores** – e.g., MongoDB, CouchDB.
    - Store semi-structured JSON-like documents.
    - Best for flexible schemas, hierarchical data.
3. **Column-Family Stores** – e.g., Apache Cassandra, HBase.
    - Data stored in columns rather than rows.
    - Best for analytical queries on large datasets.
4. **Graph Databases** – e.g., Neo4j, Amazon Neptune.
    - Data modeled as nodes and relationships.
    - Best for social networks, recommendation engines.

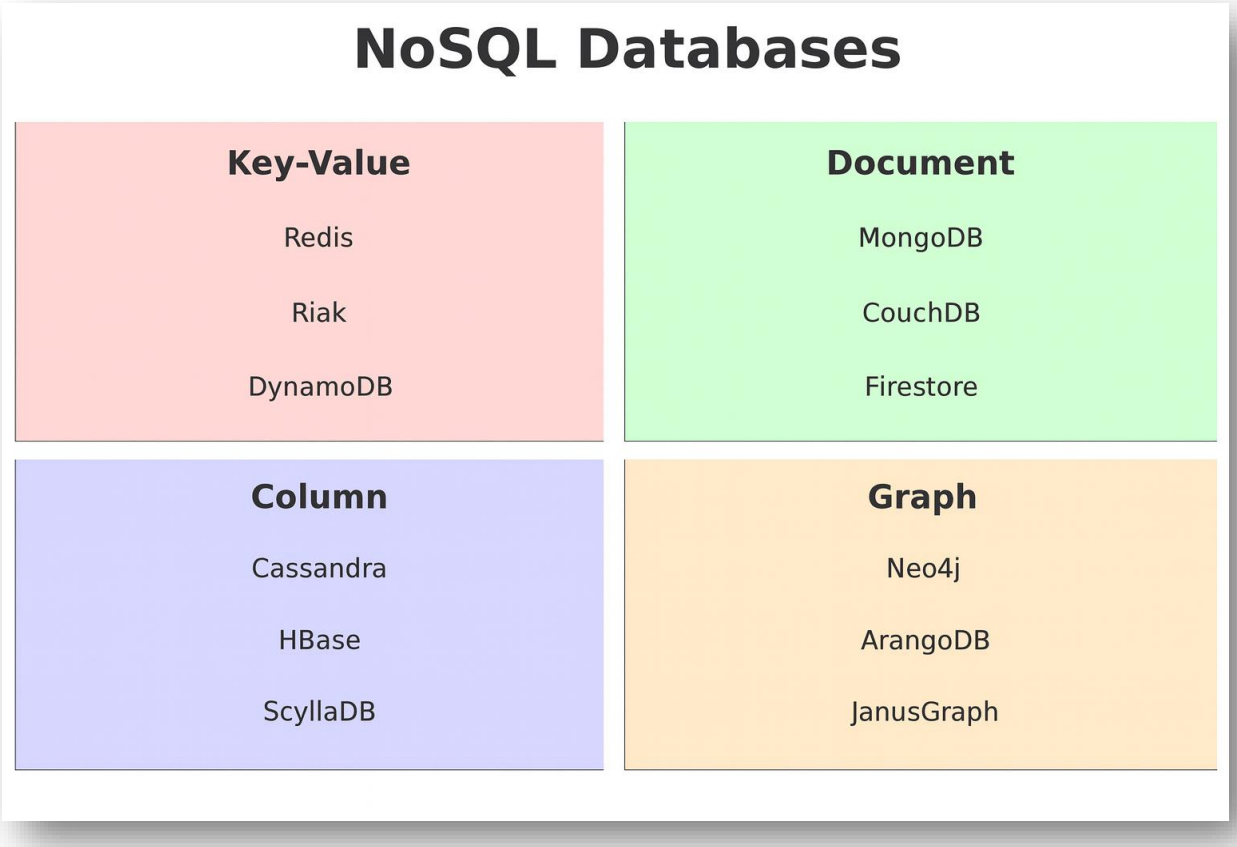
CAP Theorem

- A distributed system can only guarantee **two** of the following:
  - **Consistency** – all nodes see the same data at the same time.
  - **Availability** – every request gets a response.
  - **Partition Tolerance** – system works despite network partitions.

Integration with Big Data

- Spark connectors exist for MongoDB, Cassandra, and HBase.
- Used for hybrid pipelines: ingestion → processing → NoSQL storage → analytics.

## Types of NoSQL Databases



## Lab Exercise — MongoDB Basics

Objective:
Perform CRUD operations on a MongoDB collection.

Step-by-Step Tasks:

### 1. Start MongoDB locally or use MongoDB Atlas

```
# Local
mongod

# Atlas (example connection string)
mongodb+srv://<username>:<password>@cluster0.mongodb.net/
```

### 2. Insert documents (JavaScript)

```
db.students.insertMany([
  { name: "Alice", major: "Informatics", gpa: 3.6 },
  { name: "Bob", major: "Data Science", gpa: 3.8 }
])
```

### 3. Query documents (JavaScript)

```
db.students.find({ gpa: { $gt: 3.7 } })
```

### 4. Update a document (JavaScript)

```
db.students.updateOne(
  { name: "Alice" },
  { $set: { gpa: 3.9 } }
)
```

### 5. Delete a document (JavaScript)

```
db.students.deleteOne({ name: "Bob" })
```

## Sample Code — Spark + MongoDB Connector

**Spark + MongoDB Connector (Python)**

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MongoSparkConnector") \
    .config("spark.mongodb.input.uri", "mongodb://127.0.0.1/test.students") \
    .config("spark.mongodb.output.uri", "mongodb://127.0.0.1/test.students") \
    .getOrCreate()

# Read from MongoDB
df = spark.read.format("mongo").load()
df.printSchema()
df.show()
```

## Case Study

A telemedicine platform stores millions of daily doctor-patient chat transcripts and diagnostic images.

Big Data Approach:

- Students set up MongoDB for semi-structured chat records.
- Integrate Spark + MongoDB for text analytics (NLP on symptoms).
- Compare Cassandra for high-throughput image metadata queries.

Expected Outcomes:

Improved symptom trend analysis supports AI-driven clinical decision support systems.

## Suggested Readings

- Pramod J. Sadalage & Martin Fowler, *NoSQL Distilled*.
- MongoDB and Cassandra official documentation.
- Neo4j developer tutorials.

# MODULE 10 – CLOUD PLATFORMS FOR BIG DATA (AWS, GCP, AZURE)

## Learning Outcomes

By the end of this module, students should be able to:

- Explain why cloud platforms are widely used for big data.
- Compare the big data services of AWS, Google Cloud, and Azure.
- Deploy a simple data pipeline on a cloud platform.
- Evaluate trade-offs between on-premises and cloud-based big data solutions.
- Understand pricing models and scalability considerations.

## Theory Overview

Why Cloud for Big Data?

- **Elastic scalability** – resources scale up/down based on workload.
- **Pay-as-you-go pricing** – cost-efficient for fluctuating workloads.
- **Managed services** – abstract complexity of infrastructure.
- **Global availability** – multi-region deployments.

AWS Big Data Services

- **Amazon S3** – object storage.
- **Amazon EMR** – managed Hadoop/Spark cluster.
- **Amazon Redshift** – data warehouse for analytics.
- **Amazon Kinesis** – real-time data streaming.
- **AWS Glue** – serverless ETL service.

Google Cloud (GCP) Big Data Services

- **BigQuery** – serverless, scalable data warehouse.
- **Dataproc** – managed Hadoop/Spark clusters.
- **Dataflow** – stream & batch processing (Apache Beam).
- **Pub/Sub** – messaging and streaming.
- **Cloud Storage** – scalable object storage.

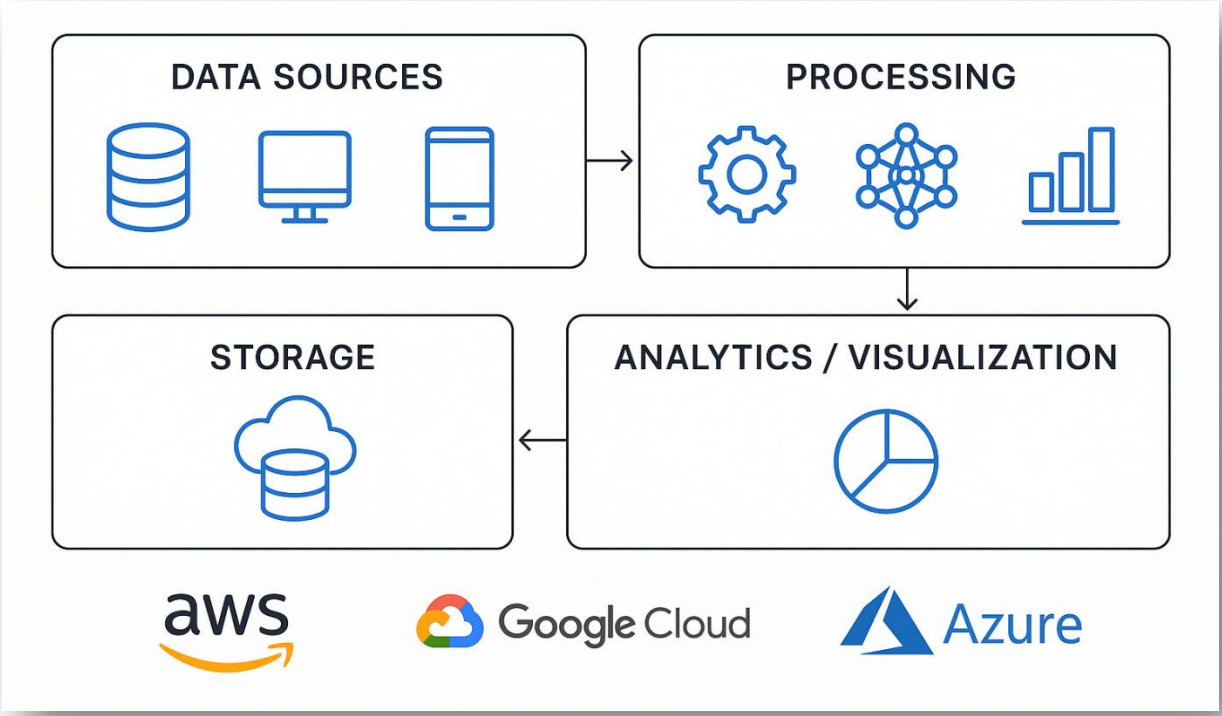
Microsoft Azure Big Data Services

- **Azure Blob Storage** – object storage.
- **HDInsight** – managed Hadoop/Spark clusters.
- **Azure Synapse Analytics** – data warehouse.
- **Azure Stream Analytics** – real-time analytics engine.
- **Cosmos DB** – globally distributed NoSQL database.

On-Prem vs. Cloud

- **On-Premises:** More control, may be cheaper at scale, higher setup costs.
- **Cloud:** Flexible, fast deployment, no upfront investment, ongoing operational cost.

## Cloud Big Data Architecture




## Lab Exercise — Querying Data on BigQuery (GCP)

Objective:

Run SQL queries on Google BigQuery.

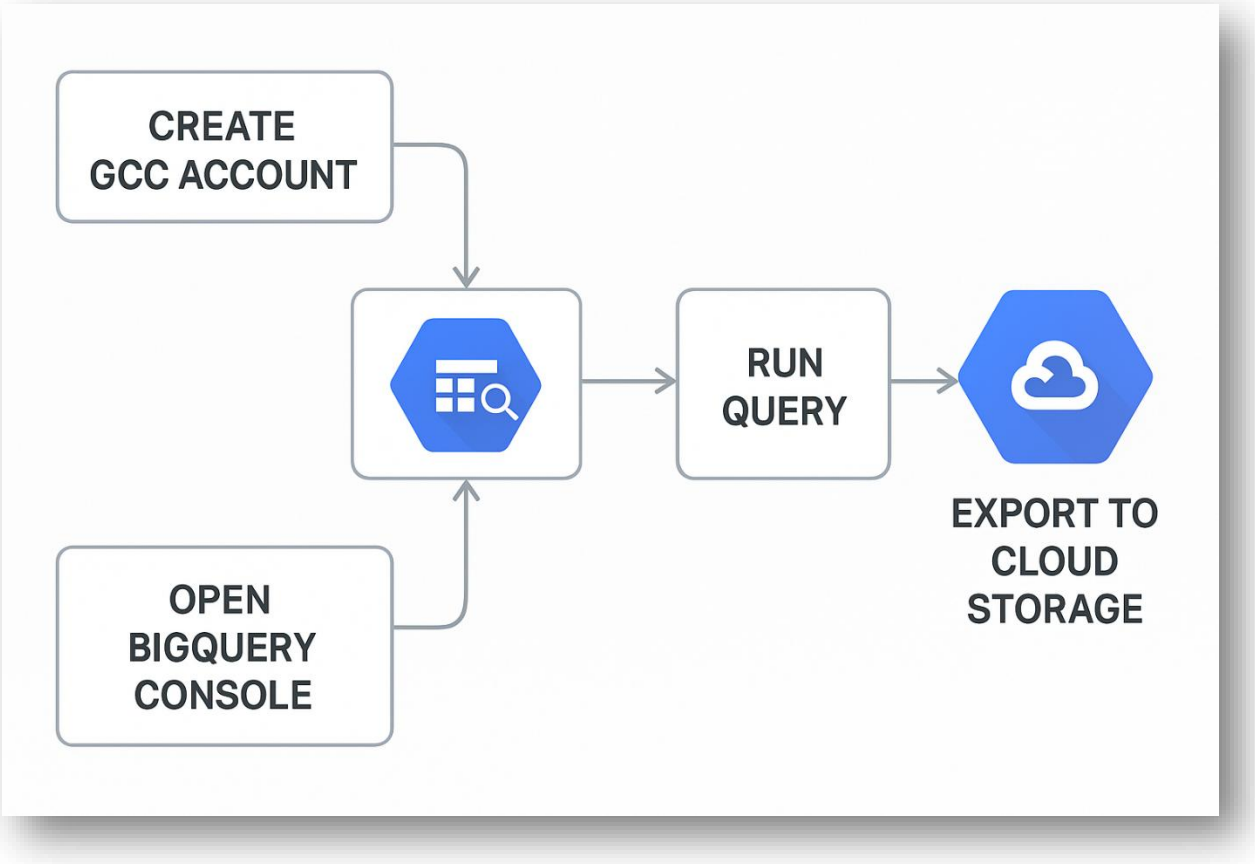
Step-by-Step Tasks

1. Create a free GCP account and open BigQuery console.
2. Load a public dataset (sql):

```
SELECT name, gender, count
FROM `bigquery-public-data.usa_names.usa_1910_2013`
WHERE state = 'CA'
ORDER BY count DESC
LIMIT 10;
```

3. Observe query execution time and cost estimate.

4. Export query results to Google Cloud Storage.




## Sample Code — AWS S3 with PySpark

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("S3Example") \
    .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem") \
    .config("spark.hadoop.fs.s3a.aws.credentials.provider",
            "com.amazonaws.auth.DefaultAWSCredentialsProviderChain") \
    .getOrCreate()

# Read CSV from S3
df = spark.read.csv("s3a://mybucket/data.csv", header=True, inferSchema=True)

df.show(5)

# Write results back to S3
df.write.mode("overwrite").parquet("s3a://mybucket/output/results")
```

## Case Study

A pharmaceutical company conducts global vaccine trial analytics across multiple countries. Data must comply with GDPR and PDP (Indonesia).

Big Data Approach:

- Use AWS S3 for raw data storage, EMR for Spark jobs, and Redshift for analytics.
- Compare with GCP BigQuery for serverless queries.
- Apply HIPAA-compliant cloud architectures.

Expected Outcomes:

Accelerated trial analytics (efficacy, adverse events) while ensuring compliance with international privacy laws.

## Suggested Readings

- AWS Big Data Services documentation.
- Google Cloud BigQuery and Dataflow guides.
- Microsoft Azure Big Data and Analytics documentation.
- Sridhar Alla, *Big Data Analytics with Azure*.

# MODULE 11 – MACHINE LEARNING WITH BIG DATA

## Learning Outcomes

By the end of this module, students should be able to:

- Explain the role of machine learning (ML) in big data analytics.
- Differentiate between traditional ML workflows and big data ML workflows.
- Apply MLlib (Spark ML) for classification, regression, and clustering tasks.
- Evaluate model performance on large-scale datasets.
- Understand scalability challenges of ML in distributed systems.

## Theory Overview

Why Machine Learning with Big Data?

- Traditional ML libraries (e.g., scikit-learn) struggle with terabyte/petabyte datasets.
- Big data ML frameworks (MLlib, TensorFlow on Spark, Mahout) distribute computation across clusters.

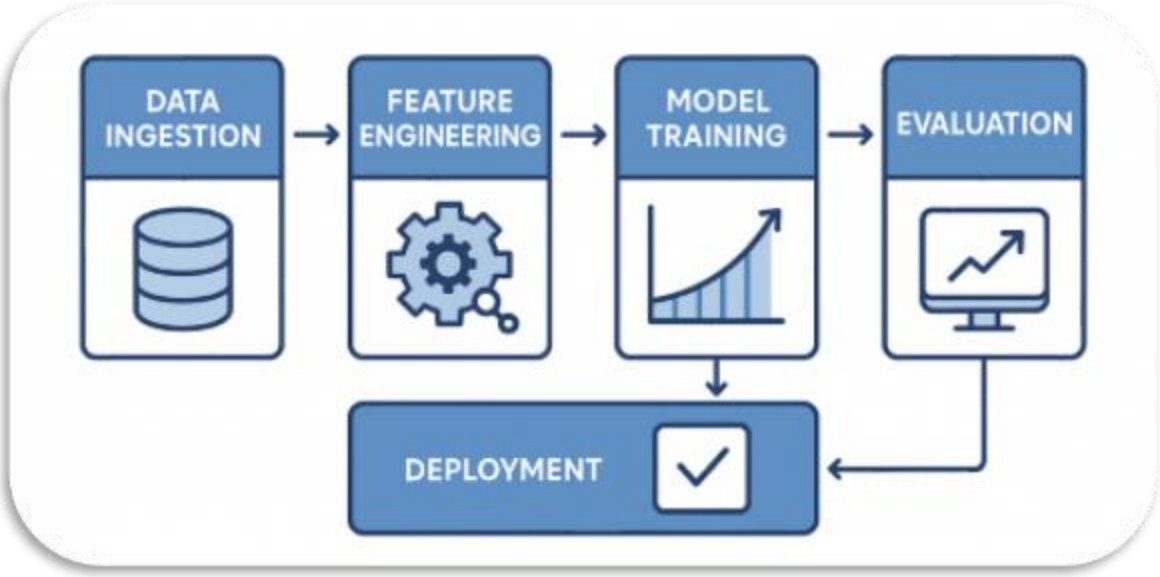
Spark MLlib

- **Features:** Scalable ML library built into Spark.
- **Supported algorithms:**
  - Classification (Logistic Regression, Decision Trees, Random Forests).
  - Regression (Linear Regression, Gradient Boosted Trees).
  - Clustering (K-Means, Gaussian Mixture Models).
  - Recommendation (ALS – Alternating Least Squares).
- **Pipelines:** Simplify preprocessing, feature engineering, training, and evaluation.

ML Workflow in Big Data

1. **Data Ingestion** – from HDFS, S3, Kafka, etc.
2. **Data Preparation** – cleaning, feature extraction, scaling.
3. **Model Training** – parallel training on distributed nodes.
4. **Evaluation** – accuracy, precision, recall, RMSE.
5. **Deployment** – batch prediction or streaming prediction.

## Big Data ML Workflow




## Lab Exercise — Logistic Regression with Spark MLlib

Objective:

Train a binary classification model on a dataset.

Step-by-Step Tasks

```python
# 1. Load dataset
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("MLlibLogReg").getOrCreate()
data = spark.read.csv("titanic.csv", header=True, inferSchema=True)
data.printSchema()

# 2. Prepare features
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=["Pclass", "Age", "Fare"],
    outputCol="features"
)
final_data = assembler.transform(data).select("features", "Survived")

# 3. Split into training and test sets
train, test = final_data.randomSplit([0.7, 0.3], seed=42)

# 4. Train logistic regression model
from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(labelCol="Survived", featuresCol="features")
model = lr.fit(train)
results = model.transform(test)
results.show()

# 5. Evaluate model
from pyspark.ml.evaluation import BinaryClassificationEvaluator, MulticlassClassificationEvaluator

# AUC
binary_eval = BinaryClassificationEvaluator(
    labelCol="Survived", rawPredictionCol="rawPrediction"
)
auc = binary_eval.evaluate(results)
print("AUC:", auc)

# Accuracy
multi_eval = MulticlassClassificationEvaluator(
    labelCol="Survived", predictionCol="prediction", metricName="accuracy"
)
accuracy = multi_eval.evaluate(results)
print("Accuracy:", accuracy)
```

## Sample Code — K-Means Clustering

```python
# 1. Import required libraries
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.clustering import KMeans

# 2. Start Spark session
spark = SparkSession.builder.appName("KMeansClustering").getOrCreate()

# 3. Load Titanic dataset (example)
data = spark.read.csv("titanic.csv", header=True, inferSchema=True)

# 4. Assemble features (using Age and Fare for clustering)
assembler = VectorAssembler(
    inputCols=["Age", "Fare"],
    outputCol="features"
)
data2 = assembler.transform(data).select("features")

# 5. Train KMeans model
kmeans = KMeans().setK(3).setSeed(42)
model = kmeans.fit(data2)

# 6. Get cluster predictions
clusters = model.transform(data2)
clusters.show()

# 7. Print cluster centers
print("Cluster Centers: ")
for center in model.clusterCenters():
    print(center)
```

## Case Study

A health insurer predicts fraudulent claims using ML. Dataset: 20M+ historical claims and hospital bills.

Big Data Approach:

- Apply Spark MLlib logistic regression for fraud detection.
- Run k-means clustering to detect unusual claim groups.
- Evaluate models using precision, recall, ROC AUC.

Expected Outcomes:

Reduce financial losses, improve fairness in claims processing, and detect systematic fraud rings.

## Suggested Readings

- Spark MLlib Documentation.
- Holden Karau, *High Performance Spark*.
- Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (for theory, though not distributed).

# MODULE 12 – DEEP LEARNING AND BIG DATA (TENSORFLOW, PYTORCH, SPARK INTEGRATION)

## Learning Outcomes

By the end of this module, students should be able to:

- Explain the role of deep learning in big data analytics.
- Differentiate between machine learning and deep learning approaches.
- Use TensorFlow and PyTorch for basic deep learning tasks.
- Integrate deep learning with Spark for distributed training.
- Apply GPU acceleration and cloud platforms for large-scale model training.

## Theory Overview

Why Deep Learning for Big Data?

- Deep learning models (e.g., CNNs, RNNs, Transformers) require large datasets to achieve high accuracy.
- Big data provides the **volume and diversity** needed for effective training.
- Requires **high-performance hardware** (GPUs/TPUs) and distributed training frameworks.

ML vs Deep Learning

- **ML:** Feature engineering is manual; algorithms like Decision Trees, SVM, Logistic Regression.
- **DL:** Features are learned automatically via neural networks.
- **ML:** Good for small/medium data.
- **DL:** Best with massive datasets.

Key Frameworks

1. **TensorFlow** – open-source DL framework (Google).
2. **PyTorch** – popular for research and production (Meta).
3. **Keras** – high-level API for DL, often built on TensorFlow.

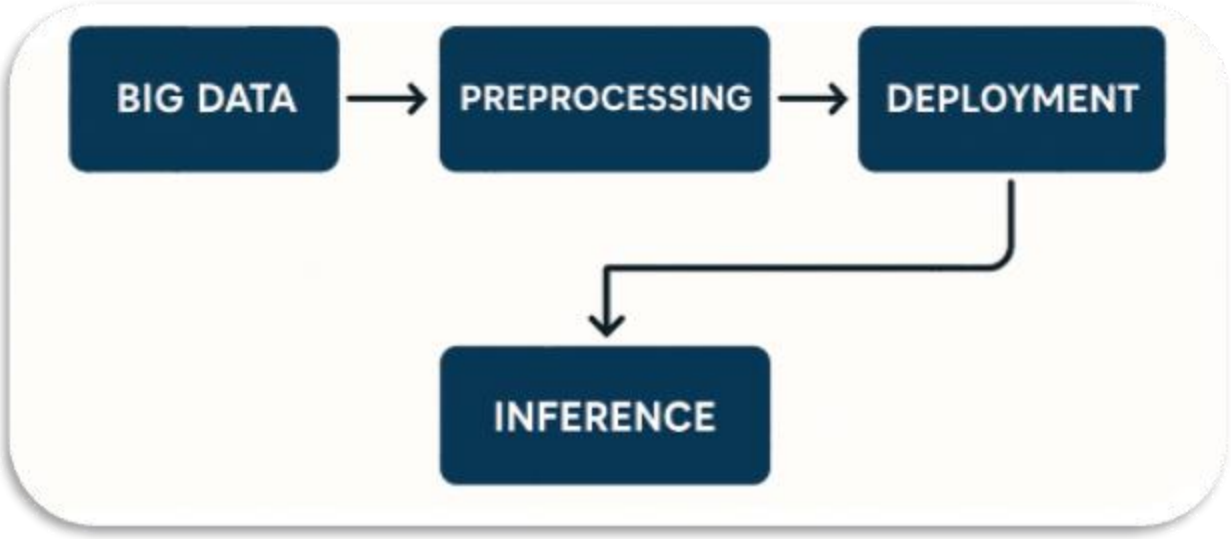
Distributed Deep Learning

- **Horovod:** Distributed training framework developed by Uber.
- **TensorFlowOnSpark / Petastorm:** Integrate Spark with TensorFlow.
- **PyTorch Lightning + Spark:** For scalable model training.

Hardware & Cloud Accelerators

- **GPUs:** CUDA cores accelerate tensor operations.
- **TPUs:** Google's specialized hardware for deep learning.
- **Cloud DL Services:** AWS SageMaker, GCP AI Platform, Azure ML.

## Deep Learning with Big Data Workflow




## Lab Exercise — Image Classification with TensorFlow

Objective:

Train a CNN to classify images using TensorFlow and integrate with Spark for preprocessing.

Step-by-Step Tasks

```
import tensorflow as tf

# 1. Load dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Reshape to include channel dimension
x_train = x_train.reshape(-1, 28, 28, 1) / 255.0
x_test = x_test.reshape(-1, 28, 28, 1) / 255.0

# 2. Build CNN model
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(10, activation='softmax')
])

# 3. Compile & train
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs=5,
                    validation_data=(x_test, y_test))
```

## Sample Code — PyTorch + Spark Integration

```python
import torch
import torch.nn as nn
import torch.optim as optim
import horovod.torch as hvd

from petastorm import make_batch_reader
from petastorm.pytorch import DataLoader
from pyspark.ml.torch.distributor import TorchDistributor


# ----------------------------
# Define a simple PyTorch model
# ----------------------------
class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(784, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x


# ----------------------------
# Training function for Horovod
# ----------------------------
def train_fn():
    # Initialize Horovod
    hvd.init()
    torch.manual_seed(42)

    # Model, criterion, optimizer
    model = SimpleNN()
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001 * hvd.size())

    # Wrap optimizer for distributed training
    optimizer = hvd.DistributedOptimizer(
        optimizer, named_parameters=model.named_parameters()
    )

    # Horovod broadcasts model & optimizer state from rank 0
    hvd.broadcast_parameters(model.state_dict(), root_rank=0)
    hvd.broadcast_optimizer_state(optimizer, root_rank=0)

    # ----------------------------
    # Load data with Petastorm
    # ----------------------------
    with make_batch_reader("file:///tmp/mnist_parquet", num_epochs=5) as reader:
        train_loader = DataLoader(reader, batch_size=32)

        for epoch in range(5):
            for batch in train_loader:
                features, labels = batch["features"].float(), batch["label"].long()

                optimizer.zero_grad()
                outputs = model(features)
                loss = criterion(outputs, labels)
                loss.backward()
                optimizer.step()

            if hvd.rank() == 0:
                print(f"Epoch {epoch+1} complete. Loss: {loss.item():.4f}")

    return model.state_dict()


# ----------------------------
# Run with Spark TorchDistributor
# ----------------------------
if __name__ == "__main__":
    TorchDistributor(num_processes=4, local_mode=False).run(train_fn)
```

## Case Study

A national cancer registry builds an AI model for radiology (X-ray, CT scans) with millions of images.

Big Data Approach:

- Train CNNs with TensorFlow + Spark integration.
- Use GPUs/TPUs for acceleration.
- Apply distributed training with Horovod.

Expected Outcomes:
Automated tumour detection reduces radiologist workload and speeds diagnostic turnaround time.

## Suggested Readings

- François Chollet, *Deep Learning with Python*.
- TensorFlow and PyTorch official tutorials.
- Uber Engineering Blog on Horovod.
- Databricks blog on Deep Learning with Spark.

# MODULE 13 – DATA VISUALIZATION AND BIG DATA DASHBOARDS

## Learning Outcomes

By the end of this module, students should be able to:

- Explain the importance of visualization in big data analytics.
- Use tools such as Tableau, Power BI, and open-source libraries for big data visualization.
- Create interactive dashboards for real-time data monitoring.
- Integrate visualization with big data backends (Spark, Hadoop, NoSQL, Cloud).
- Critically evaluate visualizations for clarity and effectiveness.

## Theory Overview

Importance of Visualization in Big Data

- Transforms raw data into actionable insights.
- Helps identify patterns, anomalies, and trends.
- Aids decision-making for non-technical stakeholders.

Challenges in Big Data Visualization

- Volume: Too much data to display directly.
- Velocity: Real-time visualization of streaming data.
- Variety: Structured, semi-structured, and unstructured data sources.

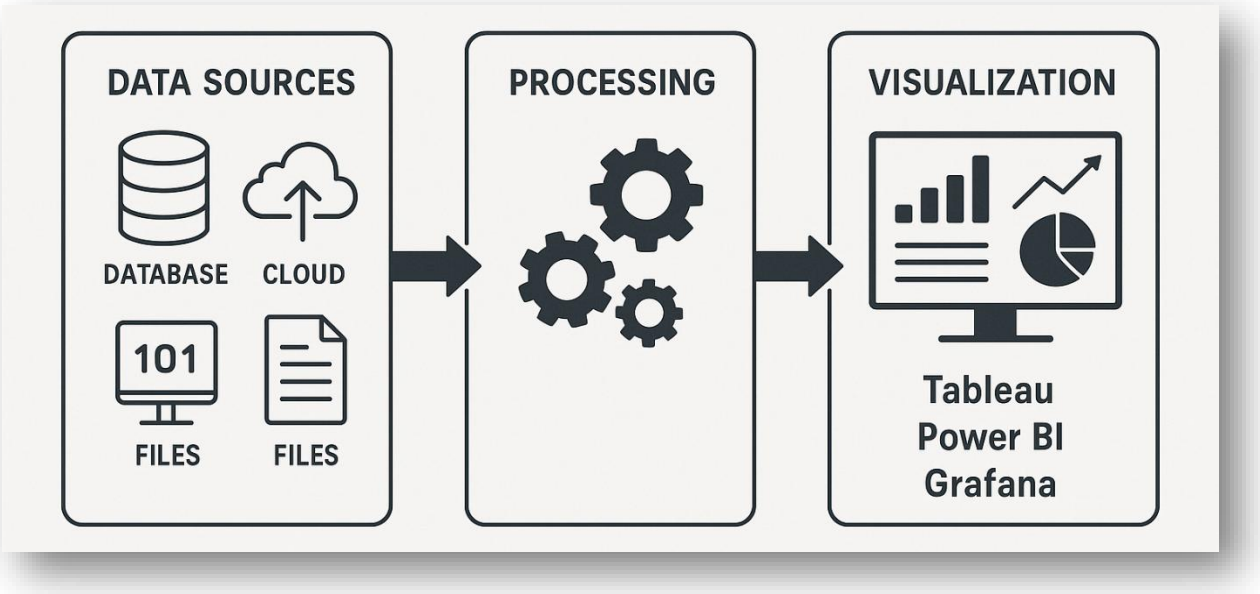
Tools for Visualization

- **Tableau** – commercial, drag-and-drop interface, integrates with Hadoop, Spark, and SQL.
- **Microsoft Power BI** – integrates with Azure, easy for business users.
- **Apache Superset** – open-source BI tool for dashboards.
- **Grafana** – popular for real-time dashboards (IoT, DevOps monitoring).
- **Python Libraries** – Matplotlib, Seaborn, Plotly for programmatic visualization.

Dashboards for Big Data

- Combine **historical analytics** with **real-time monitoring**.
- Connect to streaming sources like Kafka, Spark Streaming, or cloud services.
- Should follow **data storytelling principles**: clarity, relevance, minimal clutter.

## Big Data Dashboard Example




## Lab Exercise — Real-Time Dashboard with Grafana

Objective:
Build a live dashboard to visualize streaming data.
Step-by-Step Tasks

1. Install Grafana and InfluxDB (time-series DB).
2. Configure InfluxDB to receive IoT sensor data or simulated input.
3. Ingest sample data stream (e.g., temperature readings every second).
4. Connect Grafana to InfluxDB.
5. Build a dashboard showing:
    - Line chart (temperature over time).
    - Gauge (current temperature).
    - Alert if temperature > threshold.

## Sample Code — Python + Plotly Visualization

```python
import pandas as pd
import plotly.express as px

# Load dataset
df = pd.read_csv("bigdata_sample.csv")

# Quick preview
print(df.head())

# Scatter plot with color by education and bubble size by income
fig = px.scatter(
    df,
    x="age",
    y="income",
    color="education",
    size="income",  # bubble size by income
    hover_data=["occupation"],  # show extra info on hover if available
    title="Income vs Age by Education Level"
)

# Show interactive chart
fig.show()
```

## Case Study

A health ministry monitors COVID-19 ICU capacity across hospitals in real-time.

Big Data Approach:

- Build dashboards in Grafana/Power BI showing ICU occupancy, ventilator usage, and patient discharge rates.
- Integrate real-time feeds with Spark Streaming.

Expected Outcomes:

Data-driven allocation of ICU beds, ventilators, and staff improves pandemic crisis response.

## Suggested Readings

- Cole Nussbaumer Knaflic, *Storytelling with Data*.
- Tableau and Power BI documentation.
- Grafana Labs tutorials.
- Python Plotly and Dash official guides.

# MODULE 14 – ETHICS, PRIVACY, AND FUTURE TRENDS IN BIG DATA

## Learning Outcomes

By the end of this module, students should be able to:

- Explain ethical challenges related to big data usage.
- Understand legal frameworks for privacy and data protection (e.g., GDPR).
- Identify risks of algorithmic bias and misuse of big data.
- Discuss emerging technologies and future trends in big data.
- Propose responsible data management practices.

## Theory Overview

Big Data Ethics

- **Consent:** Individuals often unaware their data is collected.
- **Transparency:** Companies rarely disclose how data is used.
- **Bias:** Algorithms may reinforce discrimination.
- **Ownership:** Unclear who "owns" generated data.

Privacy Regulations

- **GDPR (General Data Protection Regulation – EU):**
  - Right to be forgotten.
  - Explicit consent for data collection.
  - Heavy fines for violations.
- **CCPA (California Consumer Privacy Act).**
- **Indonesia's PDP Law (UU Perlindungan Data Pribadi, 2022).**
- **The Australian Privacy Act 1988**
- **Etc.**

Risks in Big Data

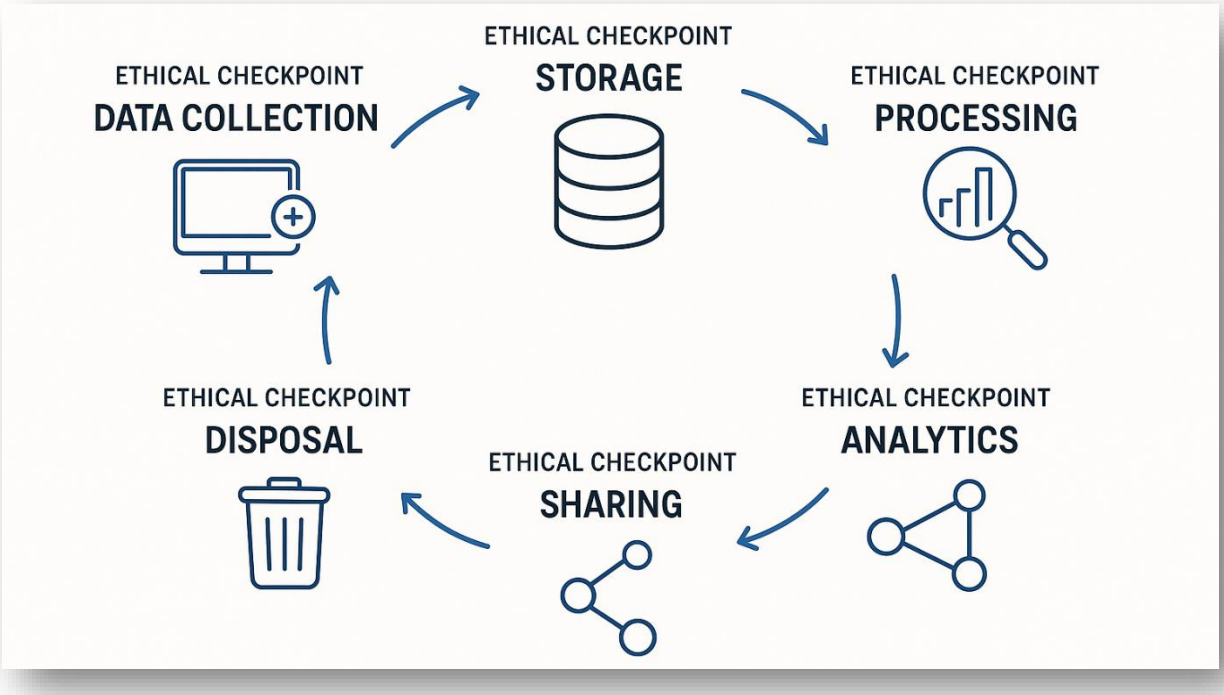
- **Surveillance:** Governments/companies monitoring citizens.
- **Discrimination:** Credit scoring, hiring algorithms, predictive policing.
- **Data Breaches:** Large-scale leaks (Yahoo, Equifax, Facebook).

Future Trends in Big Data

- **AI & ML Integration:** Automated insight extraction.
- **Edge Computing:** Processing closer to devices (IoT).
- **Quantum Computing:** Potential to revolutionize data analytics.
- **Data Democratization:** Tools accessible for non-technical users.

- **Privacy-Preserving Techniques:** Differential privacy, federated learning, homomorphic encryption.

## Responsible Big Data Lifecycle




## Lab Exercise — Data Anonymization

Objective:
Implement simple anonymization to protect sensitive information.

Step-by-Step Tasks

1. Load dataset with personal data (e.g., name, email, age).
2. Remove identifiers (name, email).
3. Apply pseudonymization (replace IDs with random codes).
4. Save anonymized dataset (phyton)

```python
import pandas as pd
import uuid

# Example dataset
df = pd.DataFrame({
    "Name": ["Ali", "Budi", "Citra"],
    "Email": ["ali@mail.com", "budi@mail.com", "citra@mail.com"],
    "Age": [25, 30, 22]
})

print("Original Dataset:\n", df)

# Remove identifiers
df = df.drop(columns=["Name", "Email"])

# Apply pseudonymization
df["UserID"] = [uuid.uuid4().hex[:6] for _ in range(len(df))]

# Save anonymized dataset
df.to_csv("anonymized_data.csv", index=False)

print("\nAnonymized Dataset:\n", df)
```

## Case Study

A biotech startup offers genomic testing services and sells insights to pharmaceutical firms. Concerns arise about data ownership, consent, and algorithmic bias.

Big Data Approach:

- Students anonymize genomic data using pseudonymization and differential privacy.
- Debate compliance with GDPR "Right to be Forgotten" and Indonesia's UU PDP Law.
- Explore federated learning as a privacy-preserving model.

Expected Outcomes:

Students understand trade-offs between innovation and ethics, ensuring responsible AI in healthcare.

## Suggested Readings

- Viktor Mayer-Schönberger & Kenneth Cukier, *Big Data: A Revolution That Will Transform How We Live, Work, and Think*.
- Shoshana Zuboff, *The Age of Surveillance Capitalism*.
- EU GDPR official guidelines.
- Research papers on differential privacy and federated learning.

# GRADING SYSTEMS

## Weighting Mark

| *MODULE TOPIC* | *SC* | *WEIGHTED MARK (%)* |
|---|---|---|
| *Introduction to Big Data* | 1 | 10 |
| *Big Data Ecosystem and Architectures* | 1 | |
| *Data Storage and Management* | 2 | 10 |
| *Hadoop Ecosystem* | 2 | |
| *MapReduce Programming* | 3 | 15 |
| *Apache Spark Basics* | 3 | |
| *Spark SQL and DataFrames* | 3 | |
| *Spark Streaming* | 4 | 15 |
| *NoSQL Databases for Big Data* | 4 | |
| *Cloud Platforms for Big Data* | 5 | 15 |
| *Machine Learning with Big Data* | 5 | |
| *Deep Learning and Big Data* | 6 | 35 |
| *Data Visualization and Big Data Dashboards* | 6 | |
| *Ethics, Privacy, and Future Trends in Big Data* | 6 | |

## SCs and Assessment Mechanism

| *ASSESSMENT* | *SC* | FORM | *WEIGHTED MARK (%)* |
|---|---|---|---|
| *Midterm Exam* | SC1 + SC2 + SC 3 | Group Assignment: System presented in a video | 35 |
| *Project 1* | SC 4 | Group Assignment: System | 15 |
| *Project 2* | SC 5 | Group Assignment: System | 15 |
| *Final Exam* | SC 6 | Group Assignment: System presented in a video | 35 |

# BIBLIOGRAPHY

## Authoritative Texts on Big Data & Ecosystems


- Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM, 51*(1), 107–113.
- Lam, C. (2010). *Hadoop in Action*. Manning Publications.
- Lin, J., & Dyer, C. (2010). *Data-Intensive Text Processing with MapReduce*. Morgan & Claypool.
- Marz, N., & Warren, J. (2015). *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Manning Publications.
- Mayer-Schönberger, V., & Cukier, K. (2013). *Big Data: A Revolution That Will Transform How We Live, Work, and Think*. John Murray.
- White, T. (2015). *Hadoop: The Definitive Guide*. O'Reilly Media.
- Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). Spark: Cluster computing with working sets. *HotCloud '10*.


## NoSQL, Storage & Databases


- Chodorow, K. (2013). *MongoDB: The Definitive Guide*. O'Reilly Media.
- Han, J., Haihong, E., Le, G., & Du, J. (2011). Survey on NoSQL database. *2011 6th International Conference on Pervasive Computing and Applications*.
- Sadalage, P. J., & Fowler, M. (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley.
- Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). The Hadoop Distributed File System. *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies*.


## Spark & Advanced Data Frameworks


- Armbrust, M., et al. (2015). Spark SQL: Relational data processing in Spark. *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*.
- Karau, H., Warren, R., & Zaharia, M. (2017). *High Performance Spark*. O'Reilly Media.
- Meng, X., et al. (2016). MLlib: Machine learning in Apache Spark. *Journal of Machine Learning Research, 17*(1), 1235–1241.

## Machine Learning, Deep Learning & AI in Healthcare


- Aurélien Géron. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.
- Chollet, F. (2018). *Deep Learning with Python*. Manning Publications.
- Esteva, A., et al. (2019). A guide to deep learning in healthcare. *Nature Medicine, 25*(1), 24–29.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature, 521*(7553), 436–444.
- Miotto, R., Li, L., Kidd, B. A., & Dudley, J. T. (2017). Deep Patient: An unsupervised representation to predict the future of patients from the electronic health records. *Scientific Reports, 6*, 26094.


## Visualization & Dashboards


- Knaflic, C. N. (2015). *Storytelling with Data: A Data Visualization Guide for Business Professionals*. Wiley.
- McKinney, W. (2017). *Python for Data Analysis* (2nd ed.). O'Reilly Media.
- Rind, A., et al. (2013). Interactive information visualization to explore and query electronic health records. *Foundations and Trends® in Human–Computer Interaction, 5*(3), 207–298.


## Ethics, Governance & Policy


- EU. (2016). *General Data Protection Regulation (GDPR)*. Official Journal of the European Union.
- Indonesia. (2022). *Undang-Undang Perlindungan Data Pribadi (UU PDP)*.
- Price, W. N., & Cohen, I. G. (2019). Privacy in the age of medical big data. *Nature Medicine, 25*(1), 37–43.
- Zuboff, S. (2019). *The Age of Surveillance Capitalism*. PublicAffairs.


## Healthcare Informatics & Big Data Applications


- Islam, M. S., Hasan, M. M., Wang, X., & Germack, H. D. (2018). A systematic review on healthcare analytics: Application and theoretical perspective of data mining. *Healthcare, 6*(2), 54.
- Lee, C. H., Yoon, H. J., & Kang, J. (2017). Using big data analytics to design clinical decision support systems. *Healthcare Informatics Research, 23*(4), 283–288.
- Raghupathi, W., & Raghupathi, V. (2014). Big data analytics in healthcare: promise and potential. *Health Information Science and Systems, 2*(1), 3.

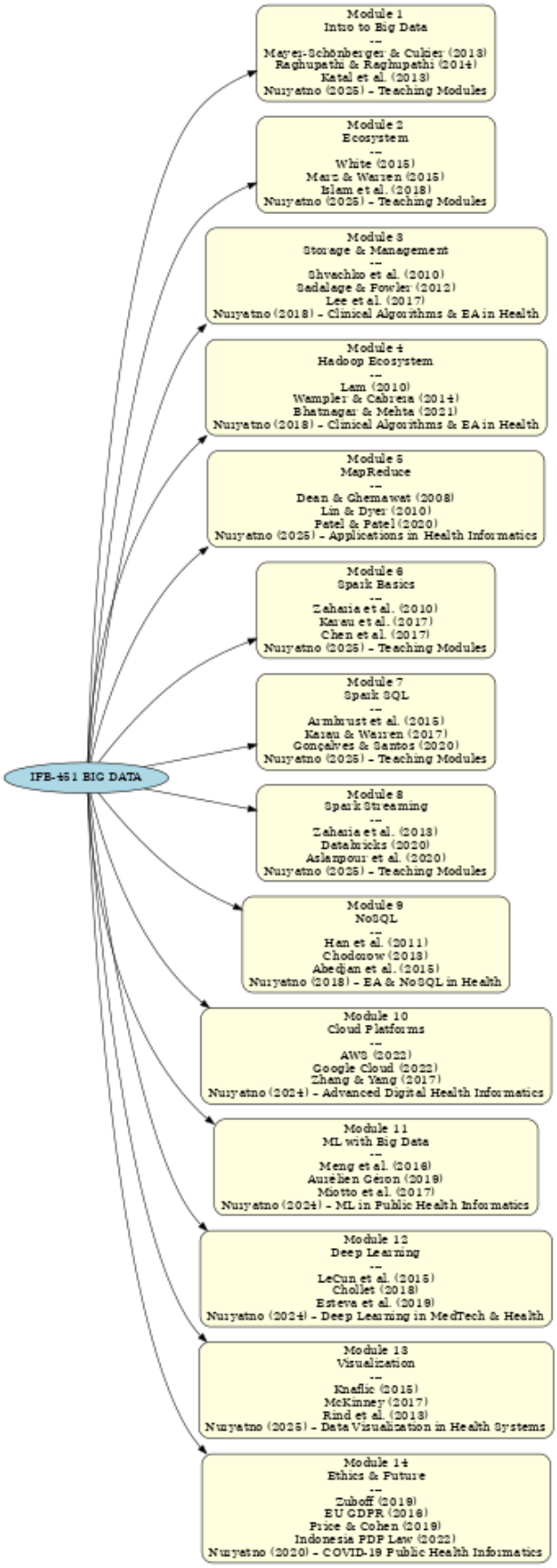
- Zhang, Y., & Yang, L. T. (2017). Real-time big data analytics for cloud-based healthcare systems. *Future Generation Computer Systems, 79*, 564–573.


## Cloud & Emerging Tech


- Amazon Web Services. (2022). *AWS Big Data Services Documentation*.
- Google Cloud. (2022). *BigQuery and Dataflow Technical Guides*.
- Microsoft Azure. (2022). *Azure Synapse and Data Analytics Documentation*.


## Dr. Edi Nuryatno Contributions


- Nuryatno, E. (2025). *IFB-451 Big Data Teaching Modules*. Institut Teknologi Nasional (Itenas), Bandung.
- Nuryatno, E. (2024). Advanced digital health informatics: Integration of AI, EA, and public health informatics. *UWA Medical School Research Award Lecture*.
- Nuryatno, E. (2020). COVID-19 informatics and public health response: Digital solutions for surveillance and mitigation. *Australian Academy of Science Expert Panel Report*.
- Nuryatno, E. (2018). Clinical algorithms and advanced ICT in health: An enterprise architecture perspective. *Proceedings of the Australasian Institute of Digital Health Conference*.


IFB-451 BIG DATA
Module 1
Intro to Big Data
---
Mayer-Schönberger & Cukier (2013)
Raghupathi & Raghupathi (2014)
Katal et al. (2013)
Nuryatno (2025) - Teaching Modules
Module 2
Ecosystem
---
White (2015)
Marz & Warren (2015)
Islam et al. (2018)
Nuryatno (2025) - Teaching Modules
Module 3
Storage & Management
---
Shvachko et al. (2010)
Sadalage & Fowler (2012)
Lee et al. (2017)
Nuryatno (2018) - Clinical Algorithms & EA in Health
Module 4
Hadoop Ecosystem
---
Lam (2010)
Wampler & Cabrera (2014)
Bhatnagar & Mehta (2021)
Nuryatno (2018) - Clinical Algorithms & EA in Health
Module 5
MapReduce
---
Dean & Ghemawat (2008)
Lin & Dyer (2010)
Patel & Patel (2020)
Nuryatno (2025) - Applications in Health Informatics
Module 6
Spark Basics
---
Zaharia et al. (2010)
Karau et al. (2017)
Chen et al. (2017)
Nuryatno (2025) - Teaching Modules
Module 7
Spark SQL
---
Armbrust et al. (2015)
Karau & Warren (2017)
Gonçalves & Santos (2020)
Nuryatno (2025) - Teaching Modules
Module 8
Spark Streaming
---
Zaharia et al. (2013)
Databricks (2020)
Aslanpour et al. (2020)
Nuryatno (2025) - Teaching Modules
Module 9
NoSQL
---
Han et al. (2011)
Chodorow (2013)
Abedjan et al. (2015)
Nuryatno (2018) - EA & NoSQL in Health
Module 10
Cloud Platforms
---
AWS (2022)
Google Cloud (2022)
Zhang & Yang (2017)
Nuryatno (2024) - Advanced Digital Health Informatics
Module 11
ML with Big Data
---
Meng et al. (2016)
Aurélien Géron (2019)
Miotto et al. (2017)
Nuryatno (2024) - ML in Public Health Informatics
Module 12
Deep Learning
---
LeCun et al. (2015)
Chollet (2018)
Esteva et al. (2019)
Nuryatno (2024) - Deep Learning in MedTech & Health
Module 13
Visualization
---
Knaflic (2015)
McKinney (2017)
Rind et al. (2013)
Nuryatno (2025) - Data Visualization in Health Systems
Module 14
Ethics & Future
---
Zuboff (2019)
EU GDPR (2016)
Price & Cohen (2019)
Indonesia PDP Law (2022)
Nuryatno (2020) - COVID-19 Public Health Informatics